

TrioTech Fusion – WRO 2025 Future Engineers Engineering Journal

Team Members : 1. Benjamin Loh De Jing
2. Evyanna Yik Jia Hui
3. Mohd Amirul Nazrin bin Mazalan
Coach : Dr. Siaw Nyuk Hiong

1. Mobility Management

1.1 Chassis Design

Our robot uses a **custom-designed LEGO SPIKE Prime chassis** optimized for stability during high-speed wall following. We selected a rectangular chassis layout because it provides a low center of gravity and enough space for the steering system, motors, and distance sensor. The layout ensures balanced weight distribution, preventing slipping during sharp turns.

The chassis was built using reinforced Technic beams to improve rigidity. Component placement was planned with the driving motor at the rear and the steering motor positioned centrally for direct control of the front wheel. This ensures smooth cornering when executing the 12 programmed right-angle turns.

1.2 Drive System

The robot uses a **45603 LEGO Technic Medium Angular Motor** to drive forward, while a **45602 LEGO Technic Angular Motor** is used to control the steering, allowing the robot to turn left and right. This design meets the “**one driving axle**” rule, as only one motor is responsible for propulsion.

Motor Selection

- **Brand:** 45603 LEGO Technic Medium Angular Motor & 45602 LEGO Technic Angular Motor
 - **Torque:** Suitable for pushing the robot at a consistent speed and
 - **Speed:** Supports high RPM to maintain the BASE_POWER of 80
- We chose this motor because it provides a good balance of torque and speed, ensuring the robot can follow walls smoothly while still having enough power to perform 12 successive turns.

1.3 Gearing System

The motor is connected directly to the drive axle without additional gears. This ensures:

- Accurate speed control
- Minimal mechanical losses
- Predictable turning speed during 90° right turns

How it Meets the Rule

Only one motor (Motor C) drives the wheels. The steering motor does not power the wheels only adjusts direction ensuring compliance.

1.4 Steering System

The robot uses Motor A as a steering actuator. It functions similarly to a servo motor and rotates to a specific angle using the `run_to_position()` command.

Steering Type

- Controlled angular steering
- PD (Proportional–Derivative) adjustment based on distance from the wall

The steering system allows rotation from -40° to $+40^\circ$, mapped as 0–360 degrees for SPIKE Prime compatibility.

1.5 Mechanical Linkage

- Motor A is connected to the front wheel via a steering beam.
- When the motor rotates, the beam shifts the front wheel left or right.
- This allows precise steering adjustments while moving at high speed.
- During turns where no wall is detected, the steering motor rotates to the aximum turn angle, enabling clean 90° cornering before returning to center.
- Motor A is connected to the front wheel via a steering beam.
- When the motor rotates, the beam shifts the front wheel left or right.
- This allows precise steering adjustments while moving at high speed.

During turns where no wall is detected, the steering motor rotates to the maximum turn angle, enabling clean 90° cornering before returning to center.

2.0 Power and Sense Management

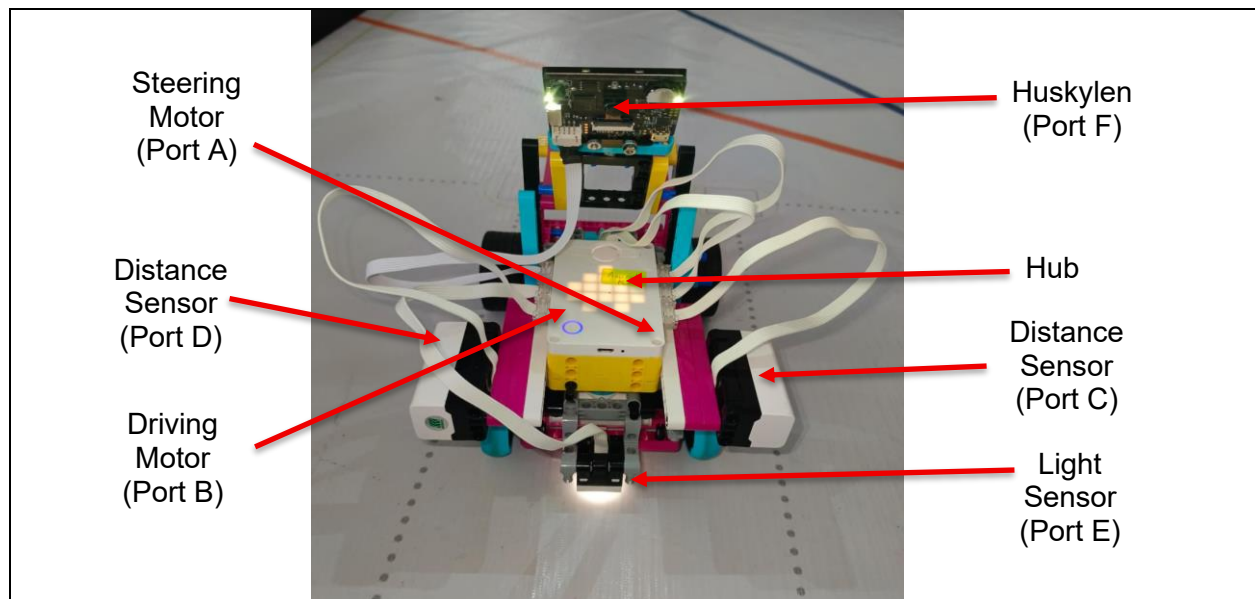
2.1 Power System

The robot uses the SPIKE Prime rechargeable battery, supplying power to:

- The hub
- Driving Motor (Port B)
- Steering Motor (Port A)
- Distance Sensors (Port C & D)
- Light sensors (Port E)
- Husklen(Port F)

Distribution Strategy

All components draw power through the SPIKE Prime hub, which regulates voltage. The robot runs at **BASE_POWER = 80** to maintain consistent speed for wall following.



Robot Wiring Diagram

2.2 Sensor System

The robot uses two Distance Sensors mounted on the left and right side of the robot. Its purpose is to maintain a target distance from the wall using PD control.

Distance and Gyro Sensor Use in Code

- The distance sensor reads distance in centimeters from the wall (during clockwise or anticlockwise movement).
- If the wall is too far or not detected, the robot performs a 90° turn.
- Gyro sensor is use to detect the current position after the 90° turn and steer the robot back to zero position.
- If the wall is detected normally, the PD algorithm calculates a steering angle.

PD Control Logic

- TARGET distance: 15 cm
- KP: 0.6 (proportional correction)
- KD: 0.25 (derivative smoothing)

This regulates steering smoothly and prevents overshooting. The robot continues wall following until 12 turns are completed.

The distance sensor's purpose is to:

- Detect the robot's distance from the right wall.
- Maintain a stable 15 cm gap using PD control.
- Detect missing walls to trigger 90° right turns.
- Count 12 successful turns as required by the challenge.

Distance Sensor Mounting Location and Reason

Location: facing sideways

- Reason:
 - The robot is program to do wall following, so the sensor must face the wall.
 - Side placement allows continuous distance reading while driving forward.
 - Ensures accurate detection of distance and corners.

How the Data Is Processed

The distance sensor provides real-time distance values (cm). The program processes this data in two ways:

1. Normal Wall-Following:
 - Distance is compared to TARGET = 15 cm
 - PD control calculates the steering angle
 - Motor A adjusts the wheel accordingly
2. Missing/Too Far Wall:
 - If distance > NO_WALL = 80 cm
 - Robot performs an automatic 90° right turn
 - A cooldown prevents double counting

This simple logic allows the robot to autonomously navigate and complete exactly 12 turns.

3. Obstacle Management

3.1. Competition Strategy

The strategy focuses on consistent wall-following and accurate corner detection rather than speed. Because the robot must complete 12 turns, reliability is prioritised over aggressive movement.

3.2 Open Challenge Strategy

- Maintain a stable distance from the inner wall to ensure smooth pathing
- Use PD control to minimise oscillation
- Use speed (BASE_POWER = 80) that is fast but still controllable

3.3 Obstacle Challenge Strategy

The left and right distance sensors are used to maintain an equal distance between the inner and outer wall while moving and navigating to avoid the green/red pillars (either

clockwise or anticlockwise). The robot is program to constantly compares these two distances and makes tiny steering adjustments to keep itself perfectly balanced in the middle, ensuring maximum stability. The data will be processed as below:

3.3.1 Calculating Steering Error

The robot must continuously read the two distance sensor values to determine the steering error:

- a. **Read Sensor Data:** Obtain real-time measurements from the left sensor () and the right sensor ().
- b. **Calculate Error:** The error calculation focuses on the difference between the two readings, indicating which side of the lane the vehicle is closer to:
 - If the error is positive, the vehicle is too far toward the inner wall.
 - If the error is negative, the vehicle is too far toward the outer wall.
 - If the error is zero, the vehicle is centered in the 1000 mm lane.

3.3.2 Applying Corrective Movement

Based on the Steering Error, the vehicle adjust its steering actuator:

- a. The robot uses **loops** and **conditionals** to repeatedly calculate the error and adjust the **steering motor**
- b. **Proportional Control:** The magnitude of the steering correction (turn angle) should be proportional to the magnitude of the error. A larger error requires a sharper correction (turning with gyro), while a small error requires minimal adjustment.
 - If (positive error), the vehicle steers proportional amounts to the **left**.
 - If (negative error), the vehicle steers proportional amounts to the **right**.

3.3.3 Stability and Robustness

For stable tracking, it is executed continuously, allowing the robot to seamlessly return to the center path immediately following the large movement required to bypass the red or green pillars.

While the distance between the track borders is consistently **1000 mm**, cornering introduces dynamic challenges. In order to handle the **four corner sections** while maintaining path balance and stability, the distance sensor data is integrated with gyro sensor data to **improve robot navigation**. On a corner section, the robot uses the gyro sensor data to initiate a precise turn angle instead of waiting for the distance sensors to record a severe deviation. During a controlled gyro-turn, the distance sensors use PD Control Logic to ensure the robot does not drift too close to the inner or outer wall.

PD Control Logic

error = distance – TARGET

derivative = error - previous_error

steer_angle = KP * error + KD * derivative

- KP (0.6): Proportional term corrects the distance
- KD (0.25): Dampens oscillation
- Steering motor rotates to adjust direction

IF distance > NO_WALL OR distance == None:

perform right turn

- The robot interprets an empty reading as an opening
- This triggers a controlled right turn
- TURN_COOLDOWN prevents repeated turns

3.4 Parking Search

The robot transitions into a search phase using Huskylen and continued navigation after it completes its three mandatory laps.

a. Identifying Parking

The vehicle uses HuskyLens to detect parking lot markers defined by **magenta** colour.

b. Pillar Management

During the subsequent route (Laps 4 and beyond) to find the parking lot, the robot no longer has to obey the red and green traffic signs for lane management. If the search detects a pillar, it executes a bypass using the check_and_bypass_pillar() function. This involves:

- i. Stopping the drive motor.
- ii. Moving backward by a set amount (e.g., 1.0 rotations) with steering centered (steering_angle=0).
- iii. Moving forward sharply with negative steering (e.g., 5.0 rotations at steering_angle=-80), executing the directional bypass.
- iv. Alignment and Positioning

Once the parking markers are detected, the robot stops its continuous navigation loop and initiates an alignment sequence:

a. **Lateral Alignment:** The robot checks the detected parking limit's horizontal position (parking_limit_x) against a defined TARGET_X_ALIGNMENT value. If the limit is outside the acceptable tolerance (X_TOLERANCE), the robot makes minor forward movements with slight steering adjustments (e.g., 0.3 rotations with steering 5 or -5) to align correctly perpendicular to the spot.

b. **Longitudinal Positioning:** The robot moves forward by a placeholder distance (e.g., 1 rotation) to ensure the rear wheels are positioned opposite the marker, ready to begin the parallel parking turn.

3.5 Execution of Parallel Parking Maneuver

Once aligned, the robot executes the maneuver using a sequenced series of time- or distance-based movements:

- i. **Initial Reverse and Turn:** The steering motor is sharply turned to a specific position (e.g., 210 degrees). The drive motor starts moving backward slowly (e.g., speed -20) for a calculated duration (e.g., 1.5 seconds). This phase tucks the rear of the vehicle into the space.
- ii. **Straightening Reverse:** The steering angle is adjusted to a different position (e.g., 150 degrees). The robot continues reversing slowly (e.g., speed -15) for another duration (e.g., 1.5 seconds) to straighten out relative to the wall.
- iii. **Final Alignment:** The steering motor is returned to the neutral position (e.g., 180 degrees).

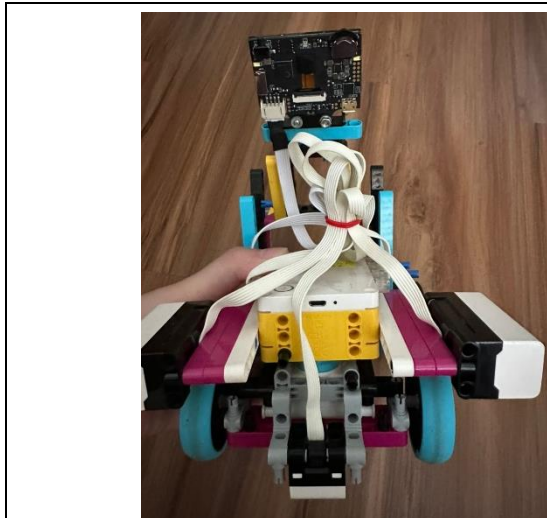
This sequence is designed to autonomously place the robot completely within the parking boundaries and parallel to the wall.

4. Pictures - Team And Vehicle

4.1 Team Photo



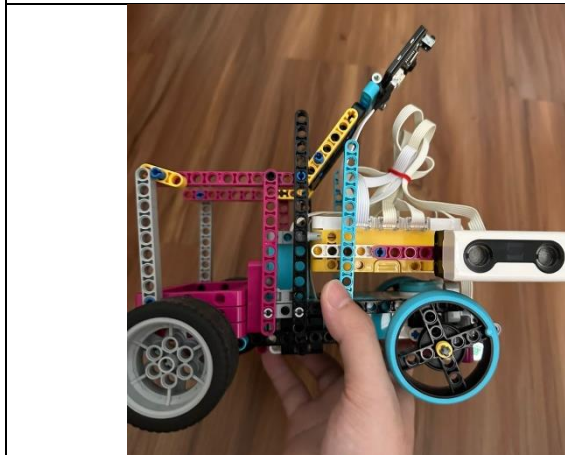
4.2 Vehicle Photo



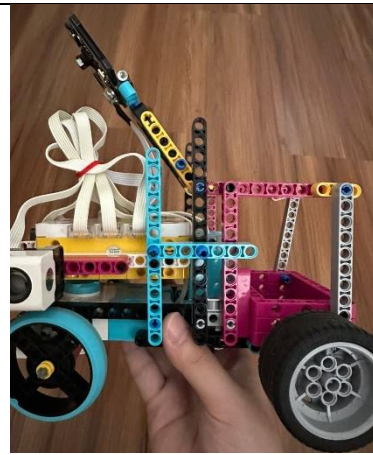
Front View



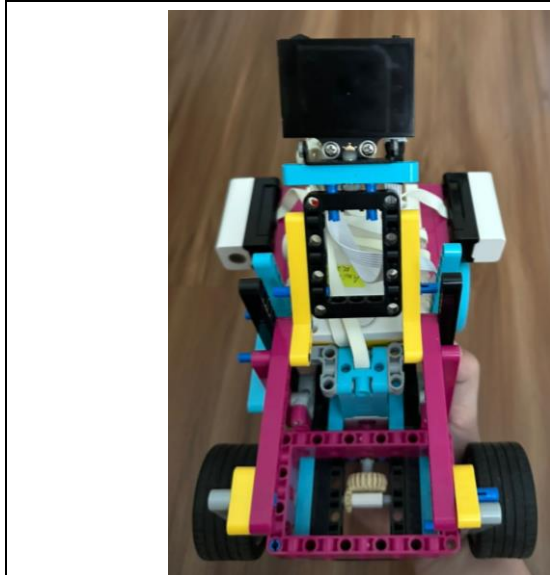
Rear View



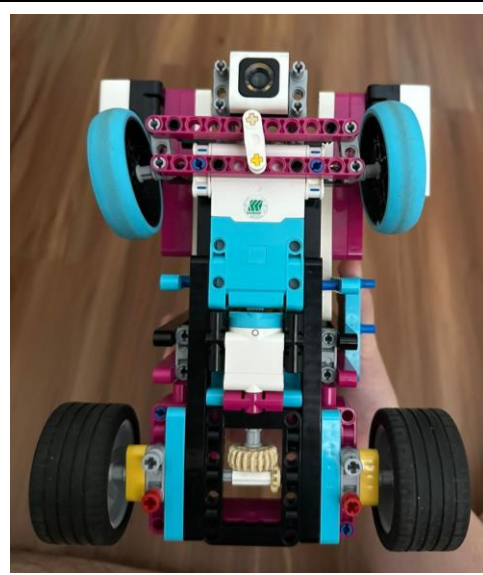
Right Side



Left Side



Top View



Bottom View

5. Performance Videos

Video 1: <https://youtu.be/1GzjrrJcUK8?si=GpolNcifah5tFU7TM>

6. GitHub Utilization

Repository Link: <https://github.com/kelabstemipgkbl-dotcom/FEC2025>

7. Engineering factor

Driving base

The basic structure of the drive base was constructed based on self-design model using Digital brick building software powered by the LEGO Group (<https://www.bricklink.com/v3/studio/download.page>). This drive base was then modified to mount the hub, distance sensors, huskylen and colour sensor to meet the needs of the robot for the competition. This robot is primarily built using the LEGO Education SPIKE Prime Set (45678) combined with a HuskyLens AI Vision Sensor. The base structure include:

- Place holder for SPIKE Prime Hub – the main programmable control unit
- Medium Angular Motor – used to drive the wheels (back)
- Technic Angular Motor – used to steer the wheels (front)

(Refer Appendix for drive base assemble)

Custom Modifications and Additions

a. Integration of HuskyLens Camera

The HuskyLens AI vision sensor is mounted on a custom-built frame at the front-top section of the robot.

- HuskyLens camera is added to enable object recognition (red/green pillars).
- The structure uses Technic beams (pink, teal, and black) to elevate the HuskyLens for a clear forward-facing view.
- The vertical sensor mast was custom-built to position the HuskyLens higher above the robot's chassis. This design improves visibility range and detection accuracy, especially when identifying objects or line colour. The adjustable mount allows for quick repositioning during testing or calibration.
- Its tilt angle is adjusted to optimize the camera's field of vision for detecting lines or objects.
- The ribbon cable connecting the HuskyLens to the main hub is routed neatly and secured using Technic pins to ensure smooth mobility and safety.

b. Distance Sensor

- Distance sensor – mounted on the side(left and right) for wall distance detection.

c. Color Sensor Function

The SPIKE Prime Color Sensor at the front of the robot is used to detect colour lines at the corner on the playfield. It reads colours to help the robot:

- Follow coloured or black lines
- Identify mission-specific colour markers
- Trigger programmed actions based on detected colours

The built-in LED supports the sensor by improving colour and contrast detection, ensuring accurate and consistent readings during missions.

d. Cable Management

All cables, especially the HuskyLens ribbon cable, are arranged neatly using Technic parts for cable guidance and securing.

- This prevents tangling or interference with moving parts.
- It also gives the robot a cleaner and more organized appearance.

Purpose of Modifications

The modifications were designed to improve the robot's autonomous navigation and vision performance. By combining the LEGO SPIKE Prime sensors with the HuskyLens AI vision system, the robot is capable of advanced functions such as object recognition and color detection. These modification enables the robot to interact with its environment more accurately and efficiently.

8. Overall Impression / Conclusion

Throughout this project, our team successfully designed and programmed a robot that can perform a series of autonomous tasks using the LEGO Education SPIKE Prime kit integrated with a HuskyLens AI vision sensor. The robot was programmed to carry out two main missions:

- a. Mission 1: Move 360° clockwise and anticlockwise to demonstrate movement control and orientation accuracy.
- b. Mission 2: Move 360° clockwise and anticlockwise at the same time detect green/red pillars using the HuskyLens and respond based on color recognition. When a red pillar is detected, the robot slightly shifts to the right and when a green pillar is detected, it shifts to the left. These movements are designed to simulate avoidance behavior rather than a complete directional change.

The biggest challenge our team encountered was handling the robot's behavior at corners. We initially relied heavily on the distance sensor to detect walls; however, at sharp corners, the sensor failed to detect nearby surfaces, causing the robot to lose its reference and perform incorrectly. To overcome this issue, we adjusted the robot's movement logic, experimented with different sensor placements that include gyro sensor and fine-tuned the distance threshold to improve detection consistency.

Through this project, our team learned the importance of sensor calibration, testing and integration between multiple inputs such as the color sensor, distance sensor and HuskyLens. We also developed a better understanding of how hardware positioning and programming logic directly affect the robot's performance in dynamic environments.

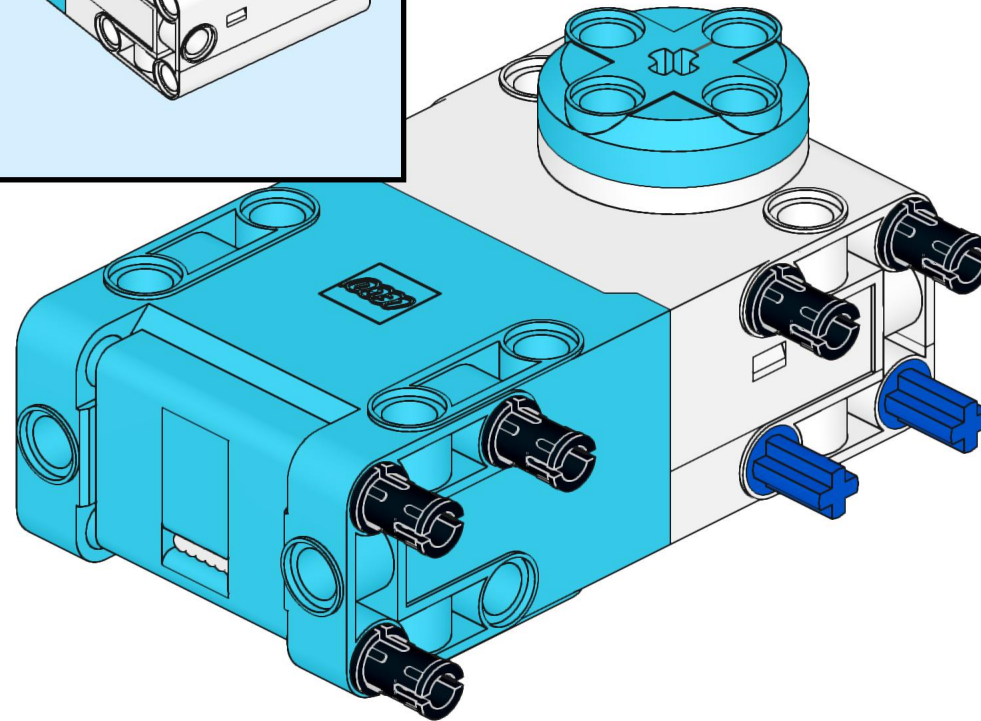
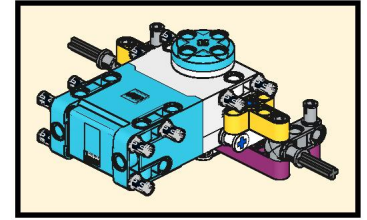
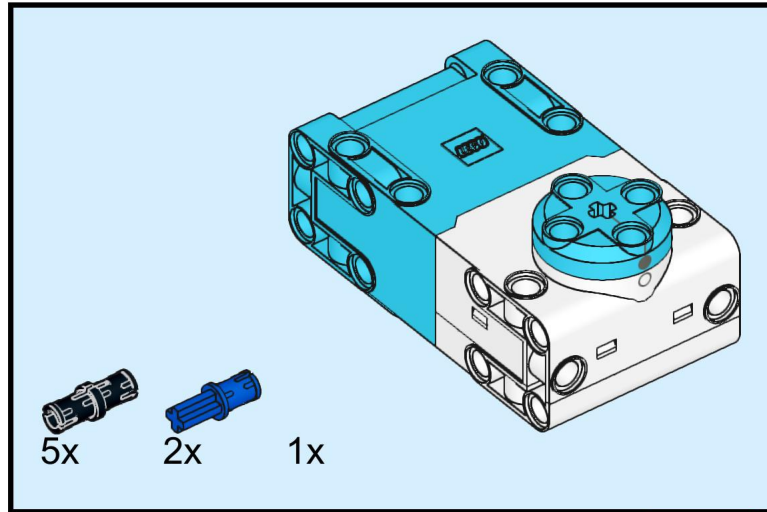
For future improvements, we plan to:

- Add additional sensors or angled mounts to improve wall detection at corners.
- Enhance the code logic to predict and correct missed detections automatically.
- Refine the robot's chassis design for more stable turning and smoother transitions between missions.

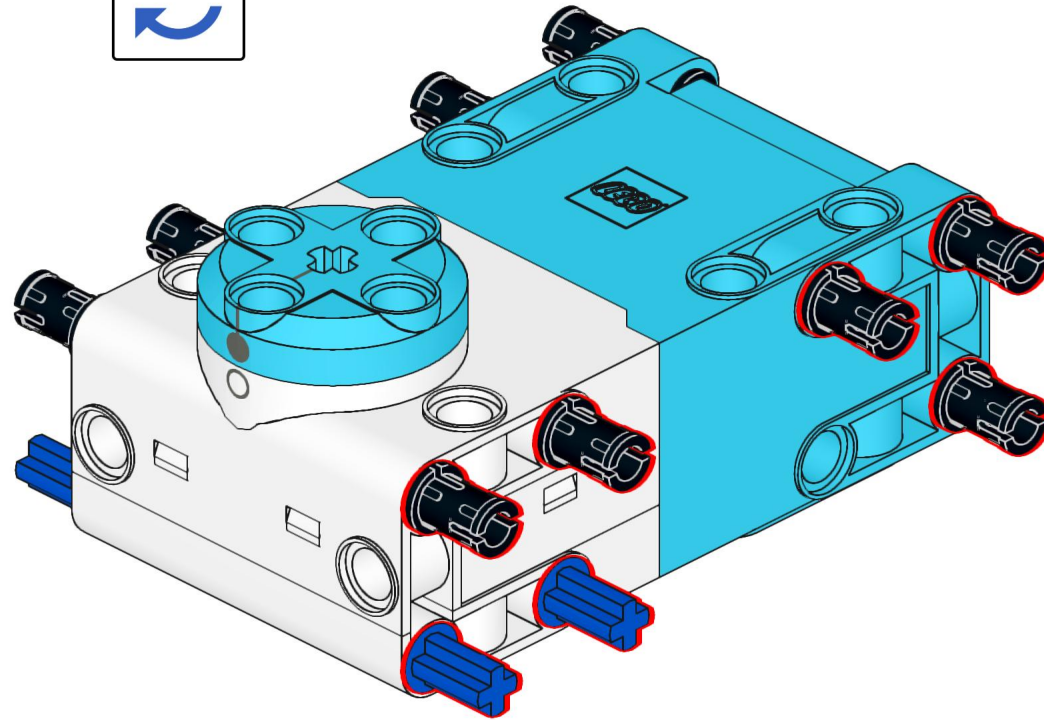
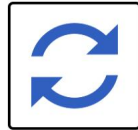
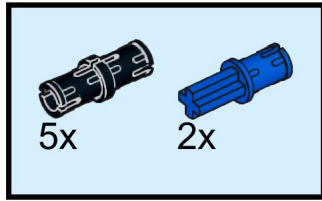
Overall, this project has been a valuable learning experience in both robotic design and problem-solving and it strengthened our teamwork, creativity and critical thinking in developing a functioning autonomous system.

Appendix: Drive Base Assembly

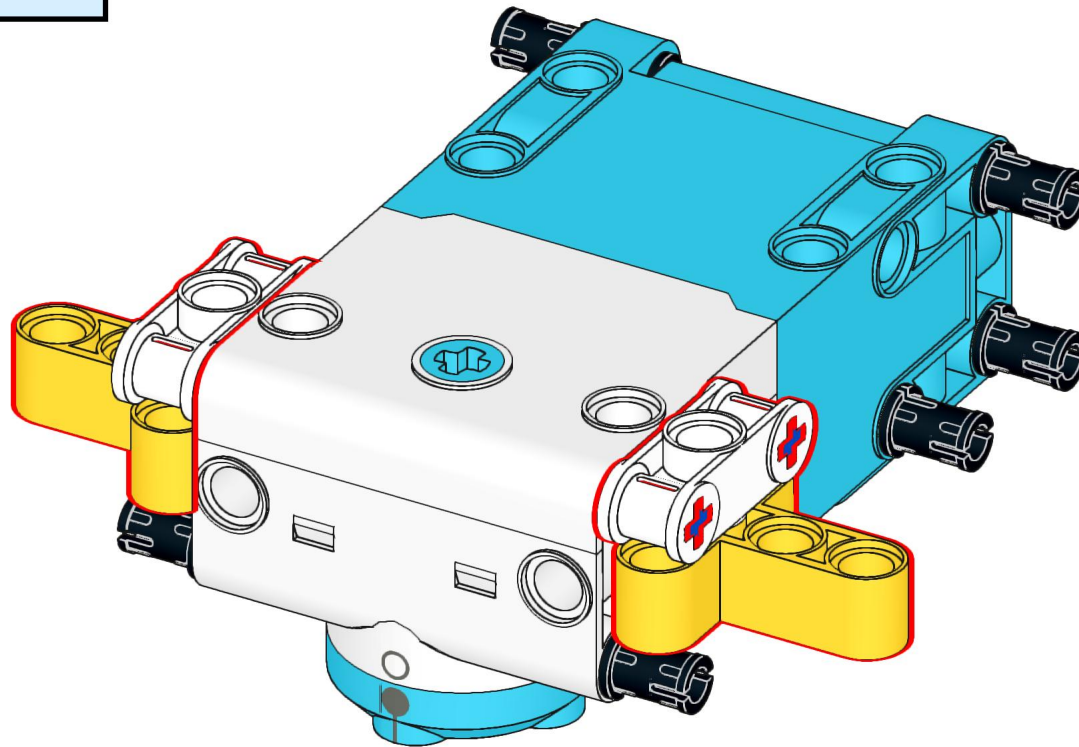
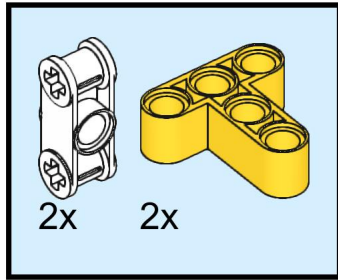
1



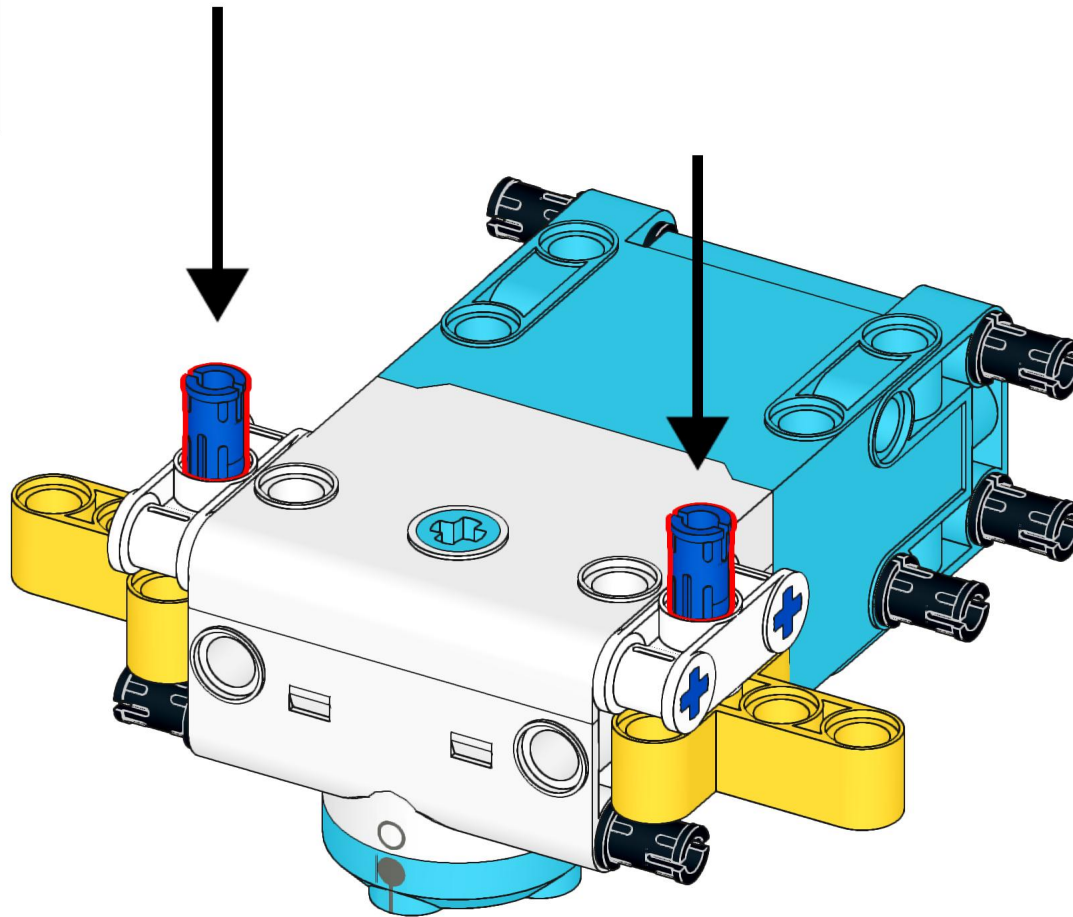
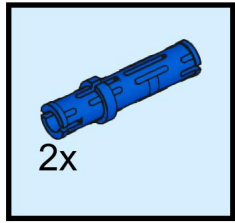
2



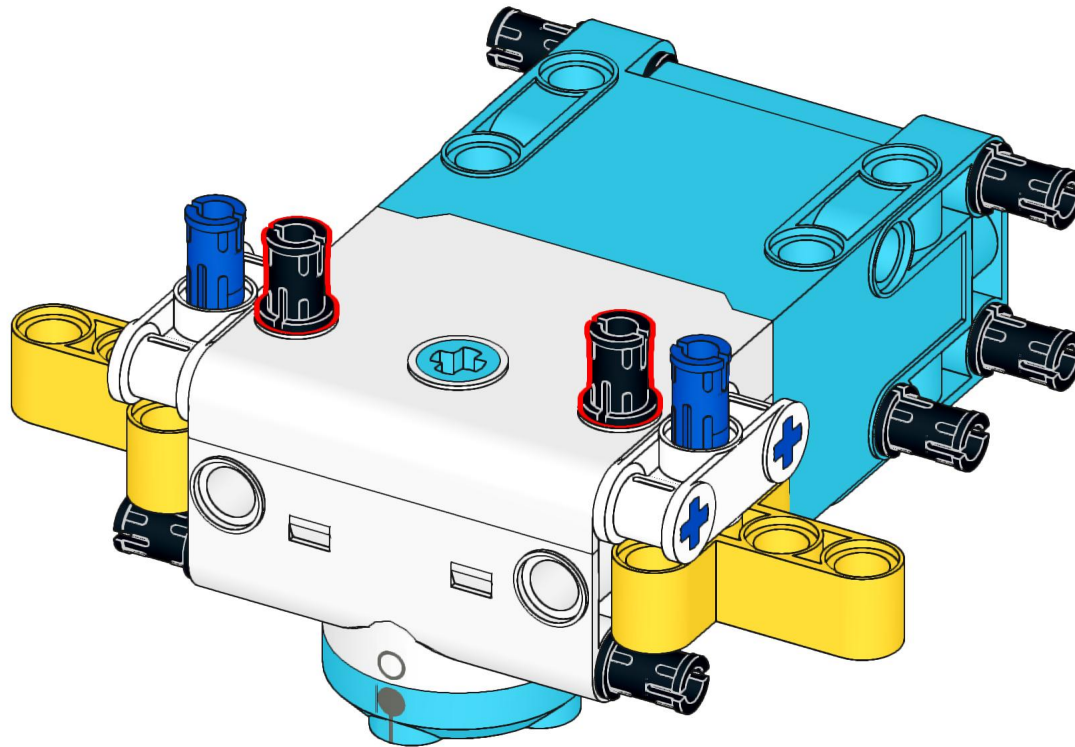
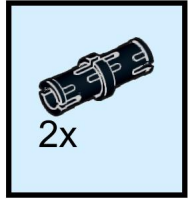
3



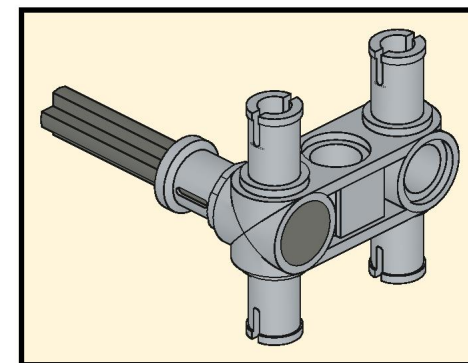
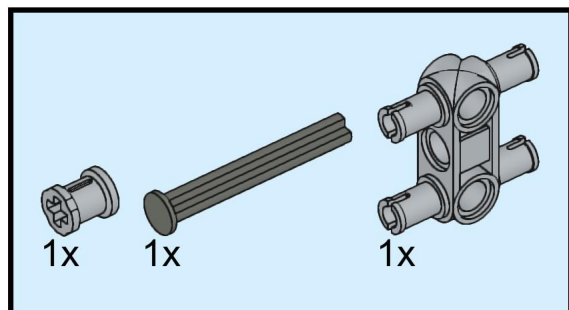
4



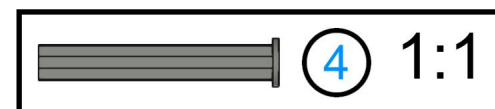
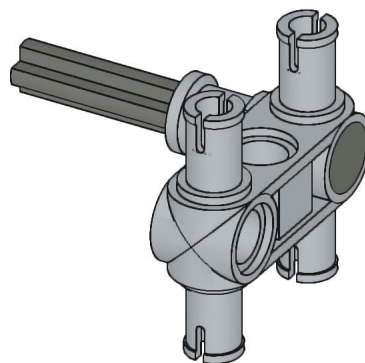
5



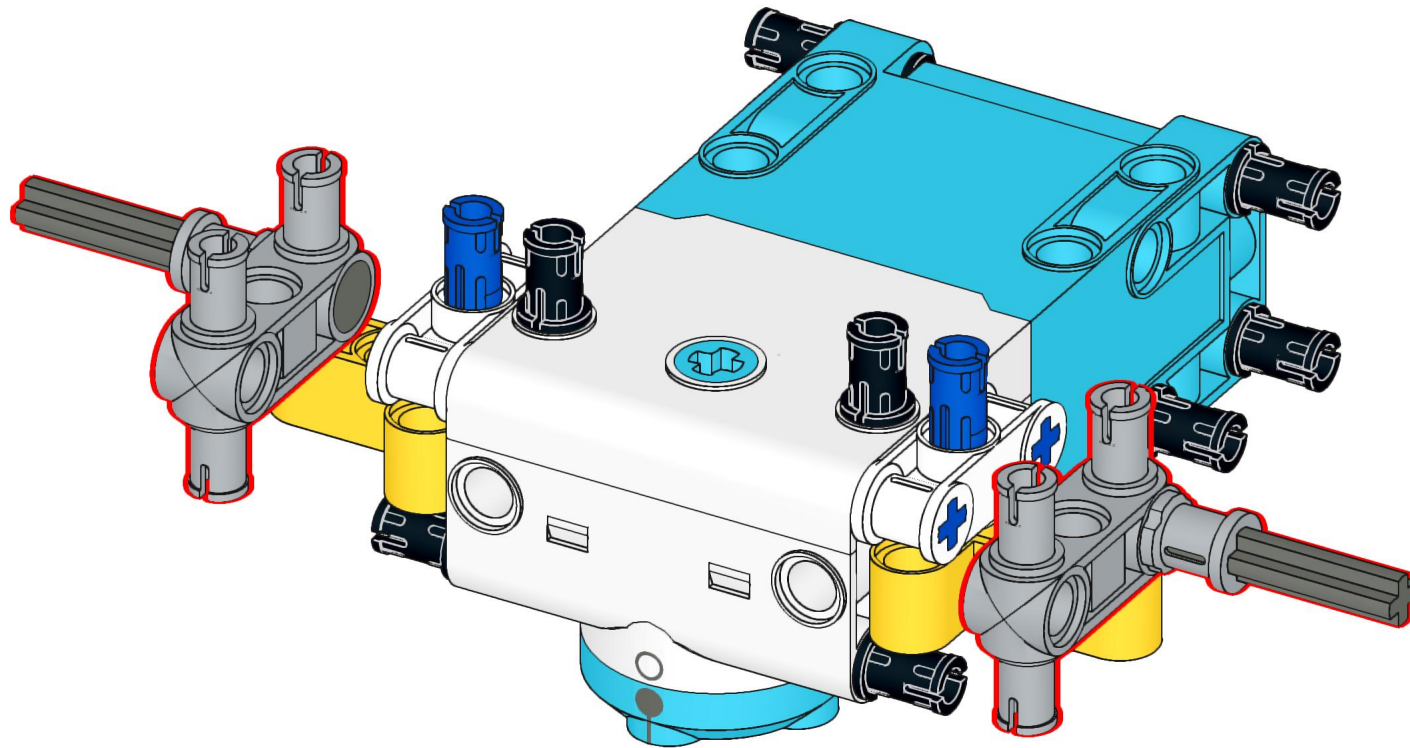
6



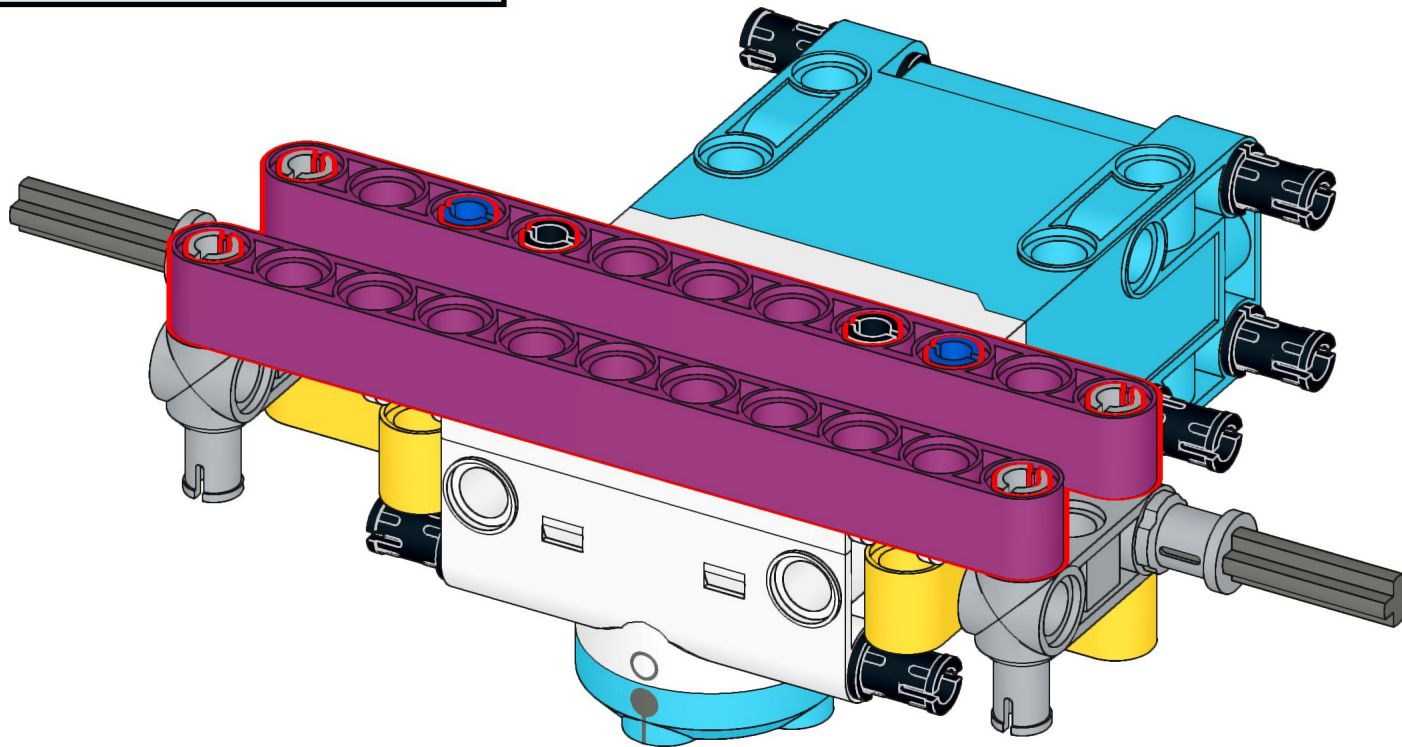
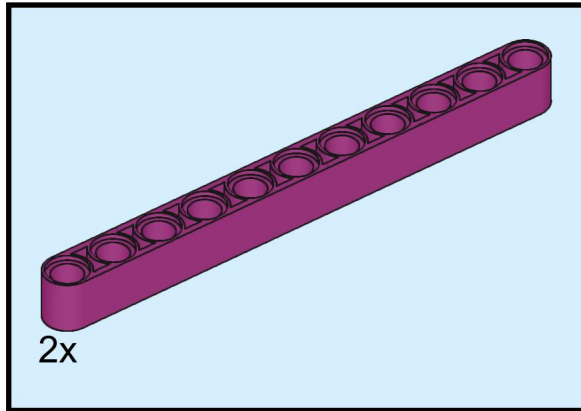
2x



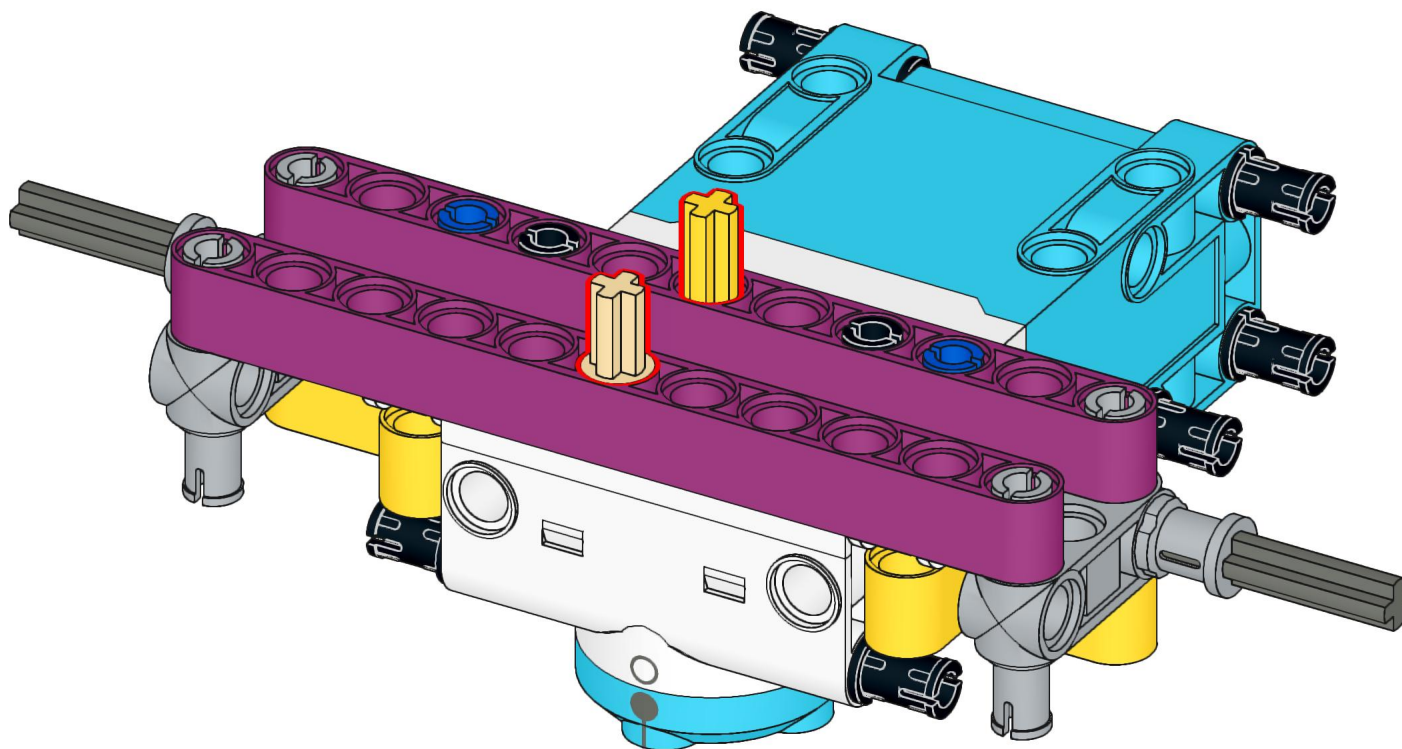
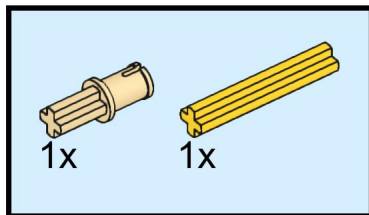
7



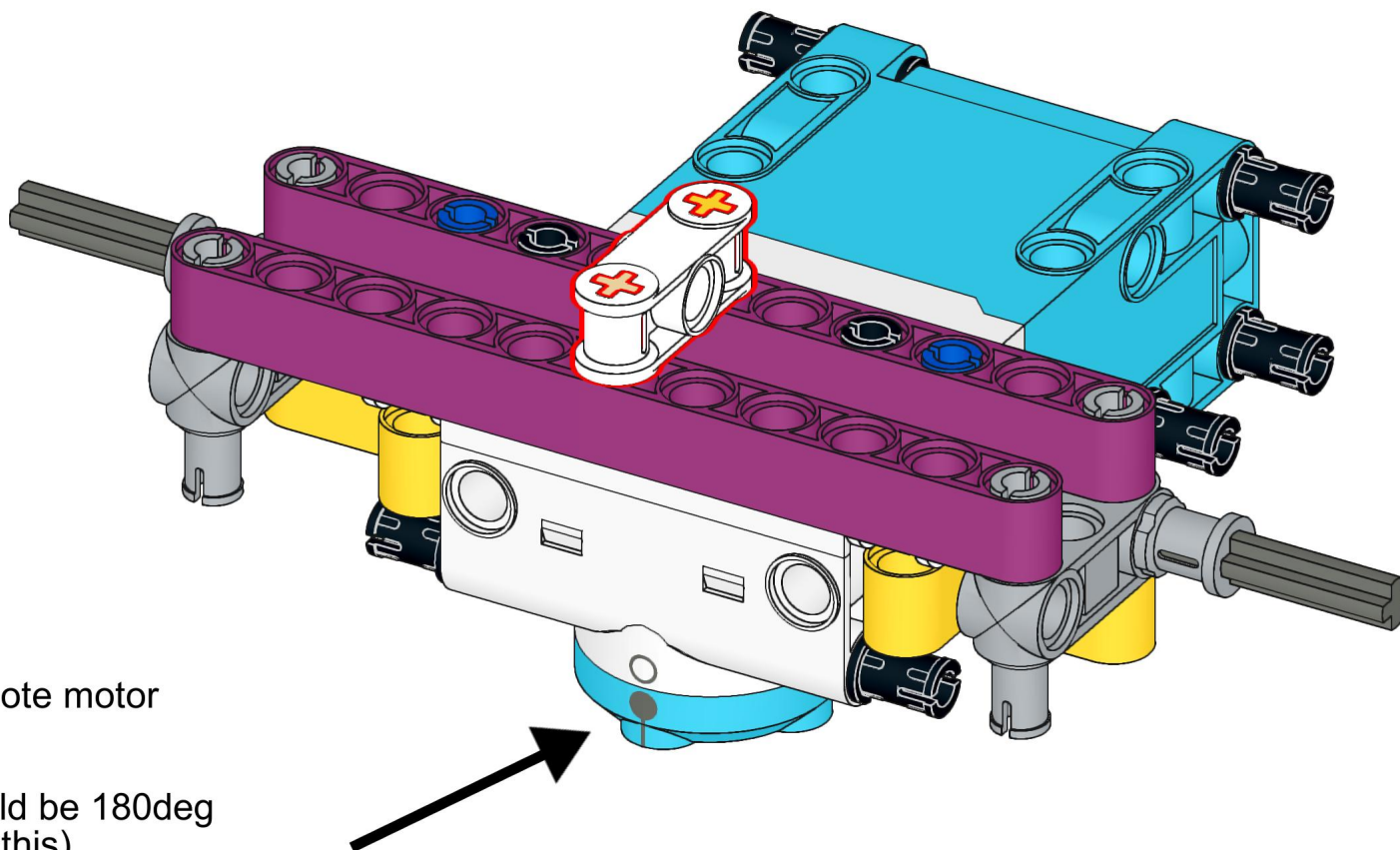
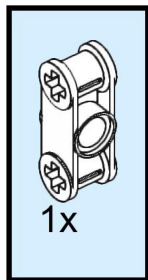
8



9



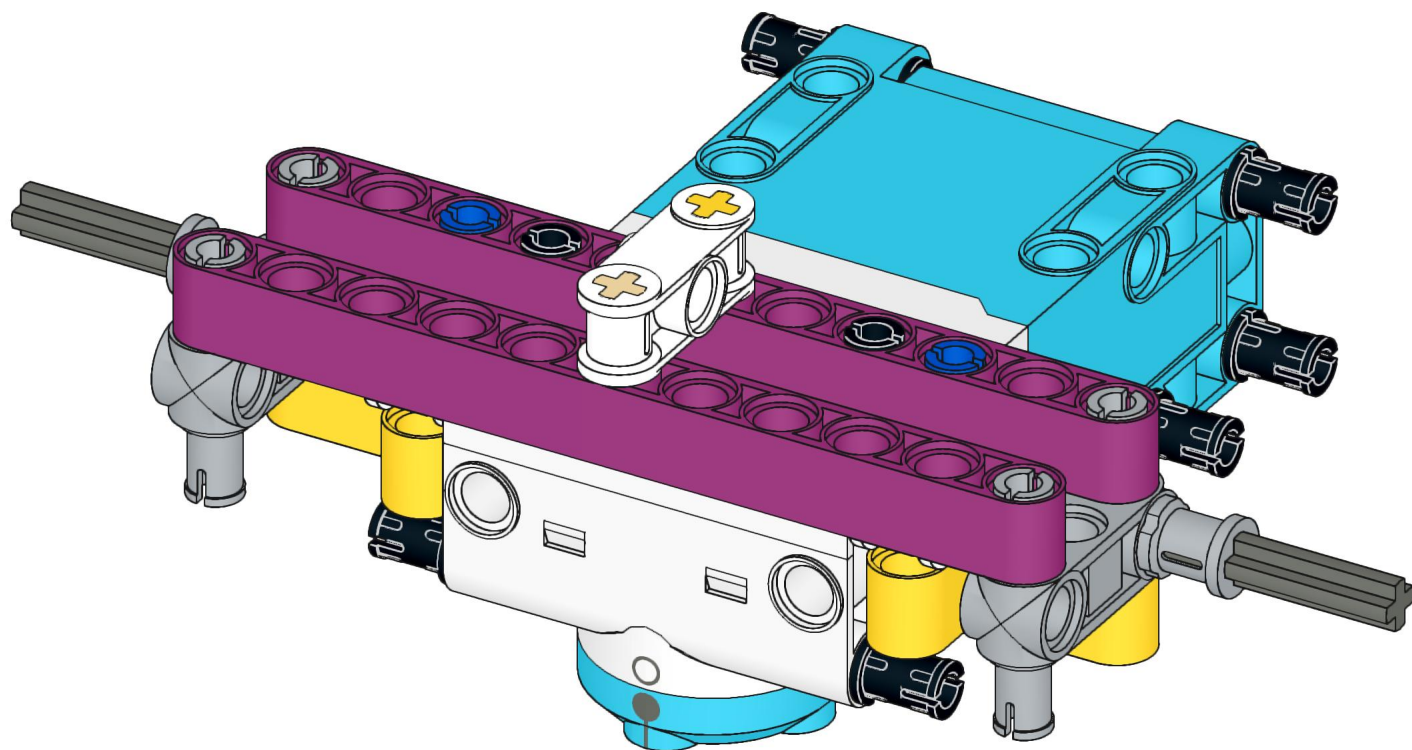
10



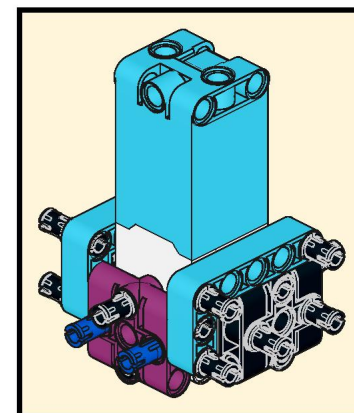
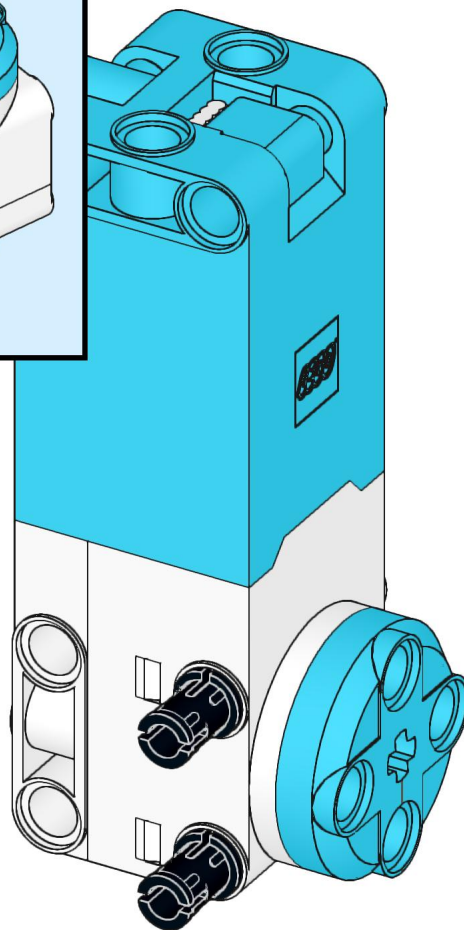
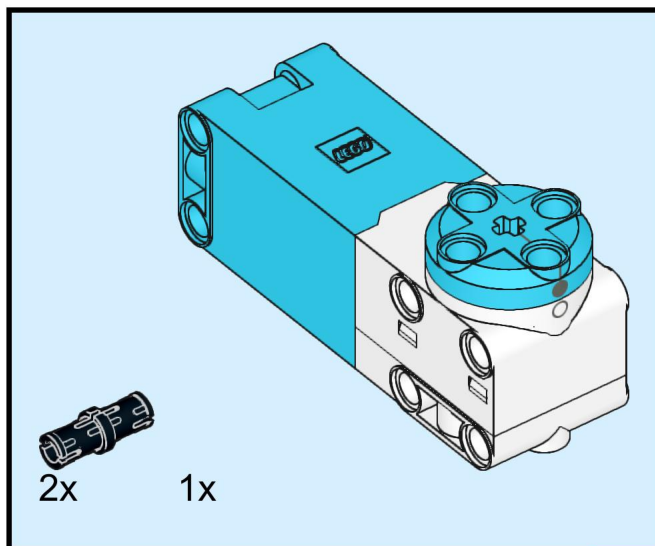
Please take note motor position

the dots should be 180deg apart(not like this)
i cant make it from studio app
sorry

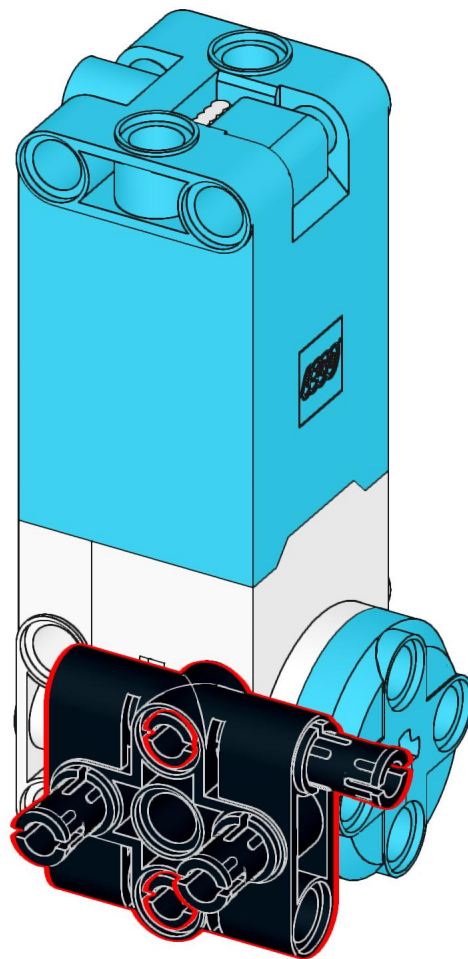
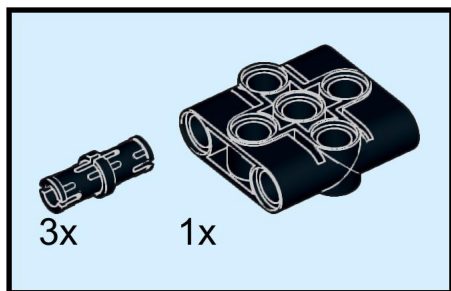
11



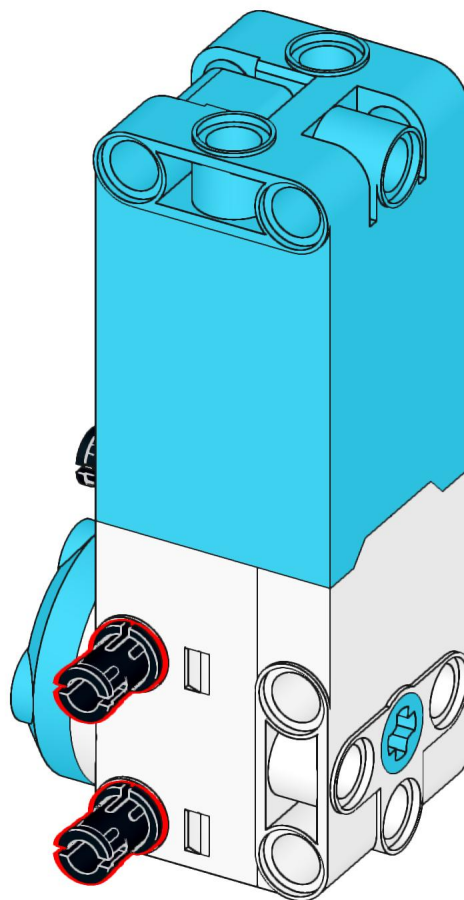
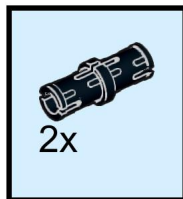
12



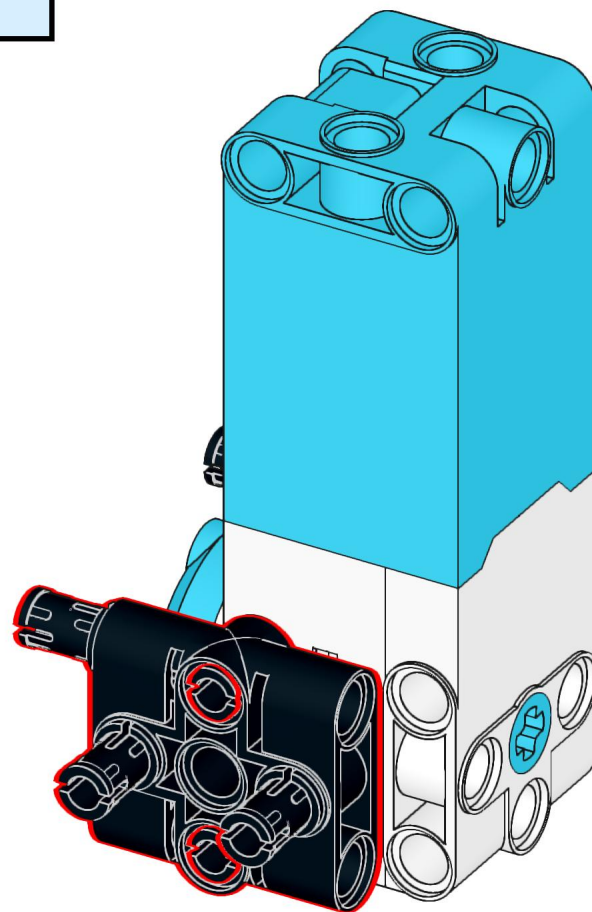
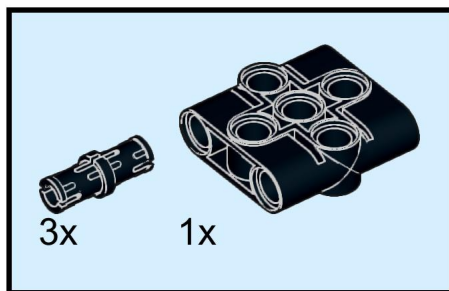
13



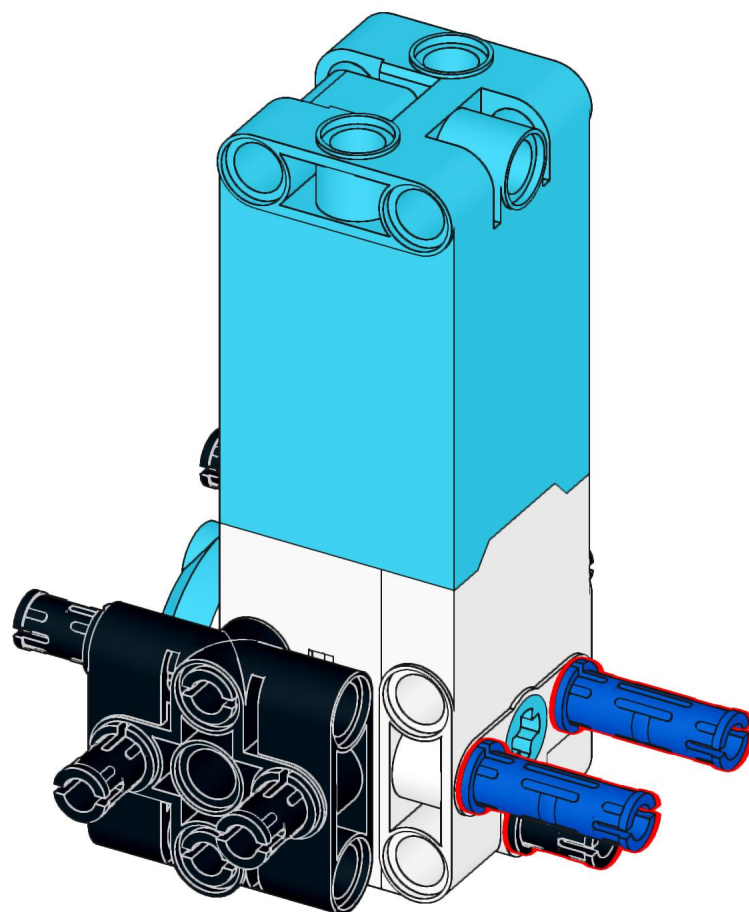
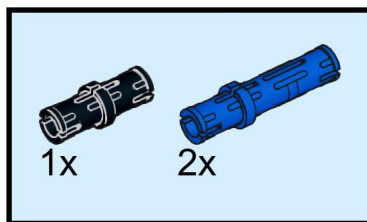
14



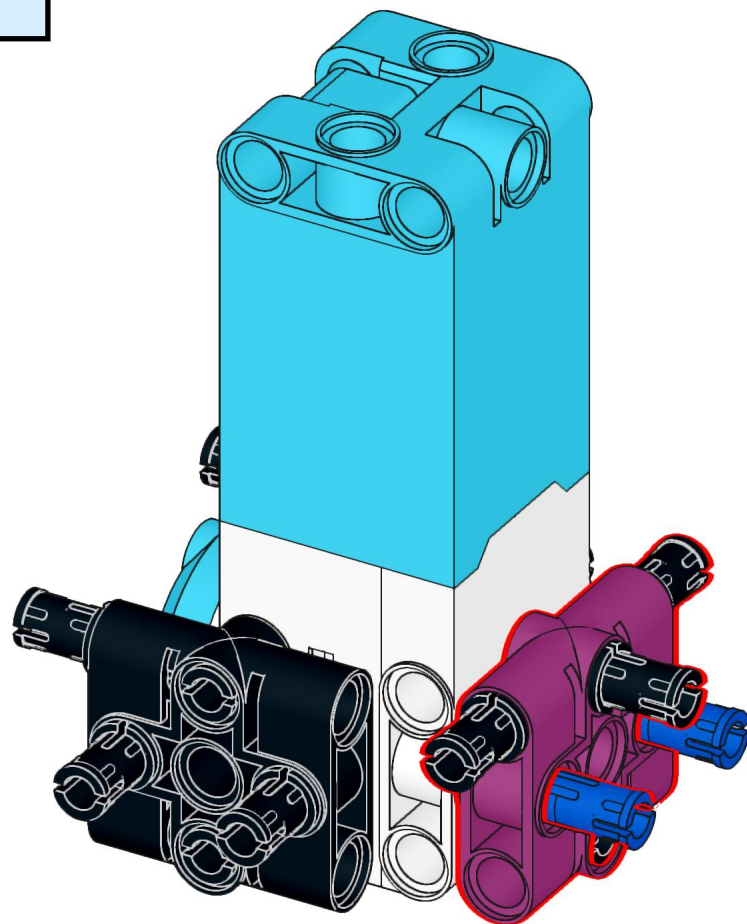
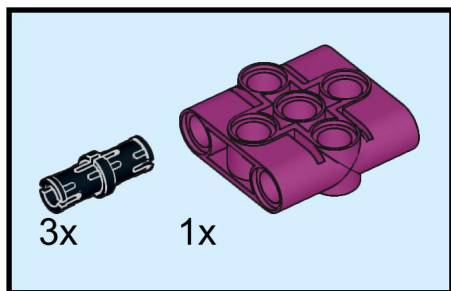
15



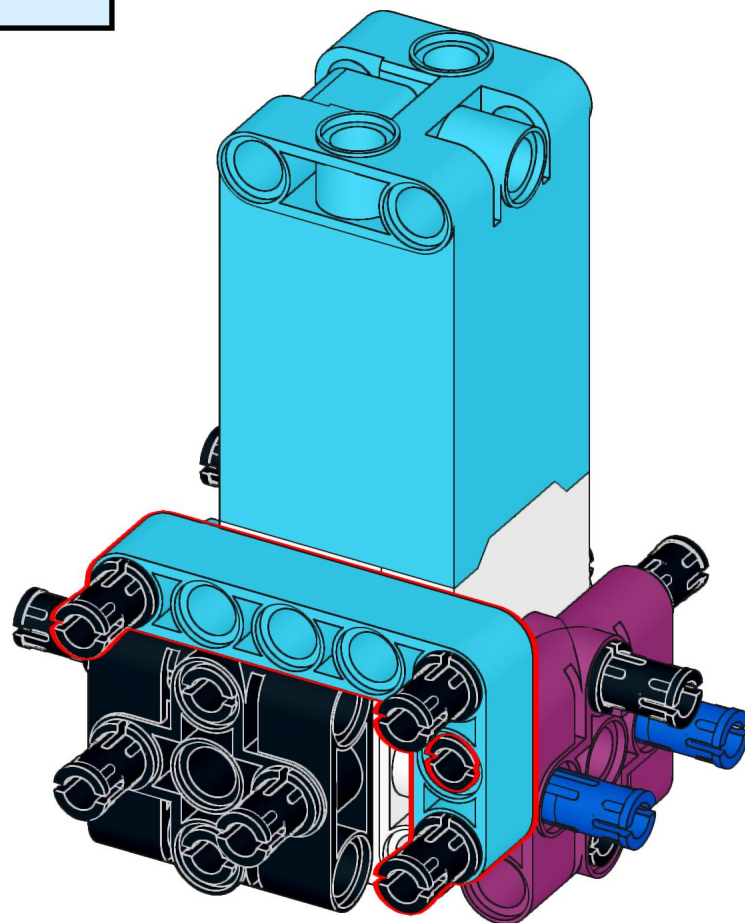
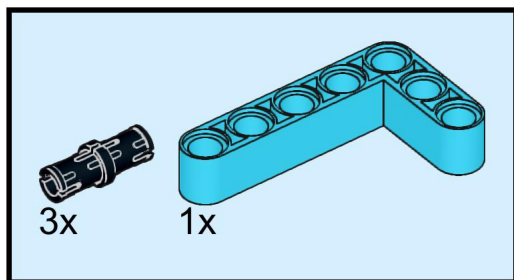
16



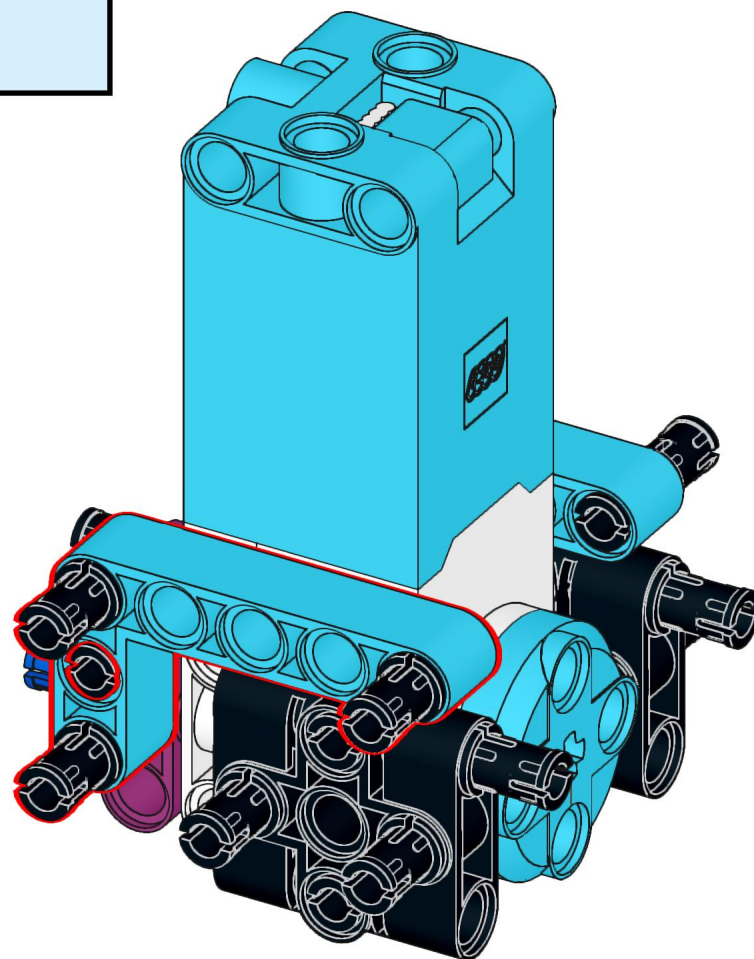
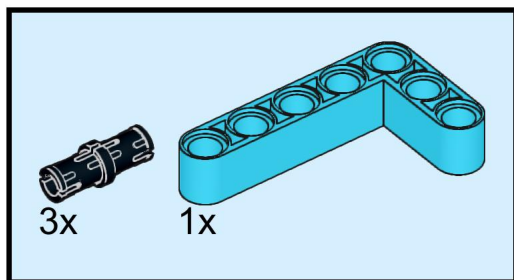
17



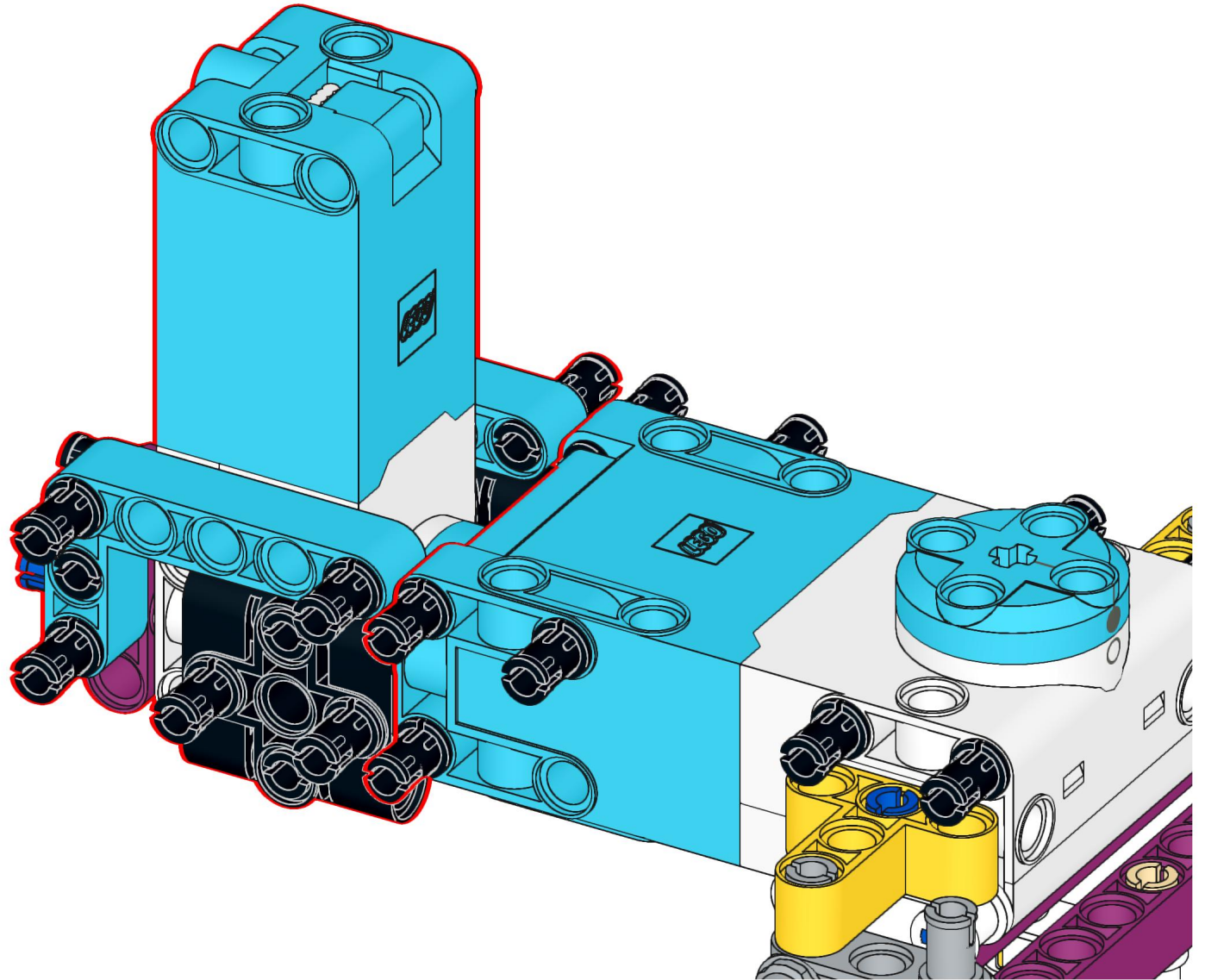
18



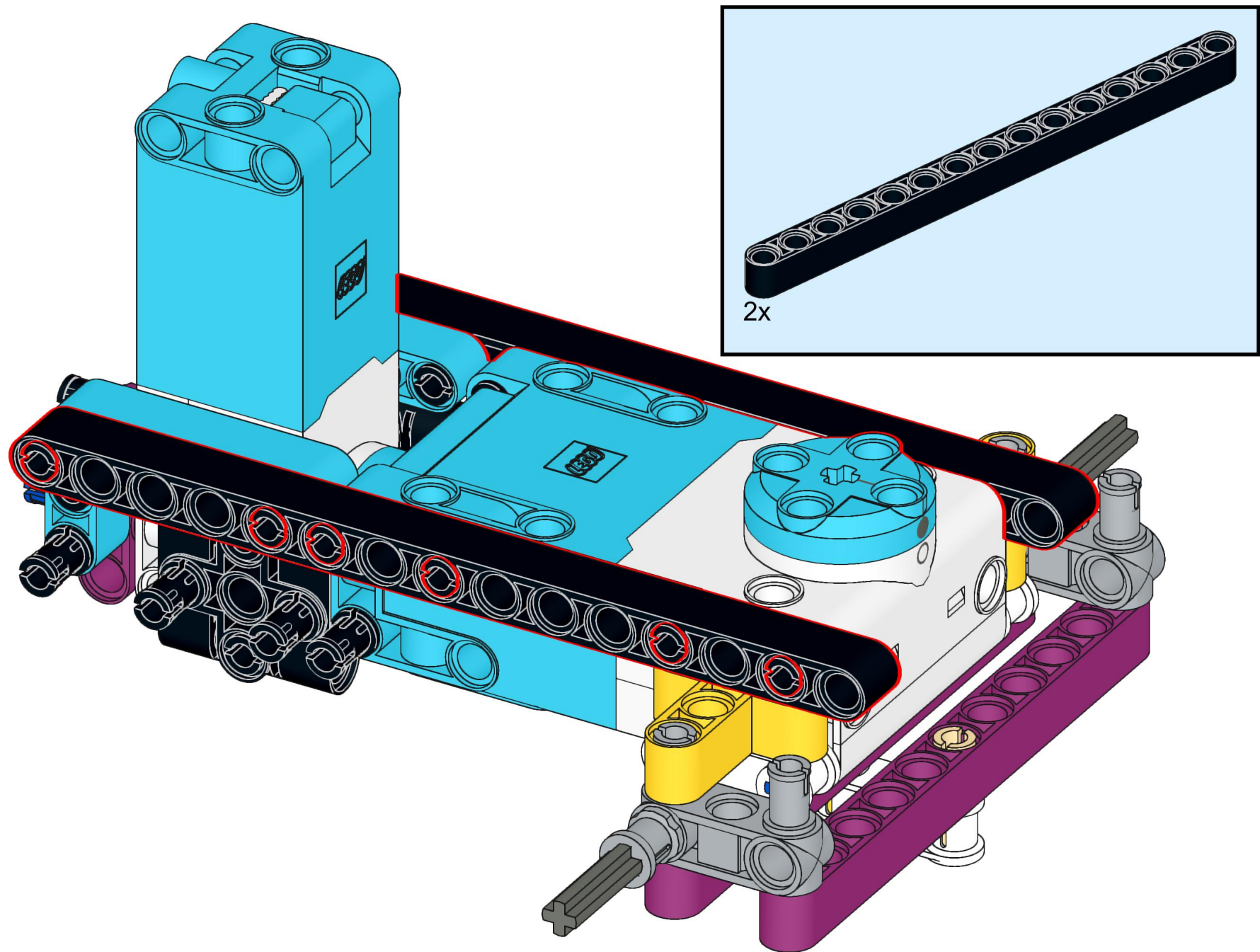
19



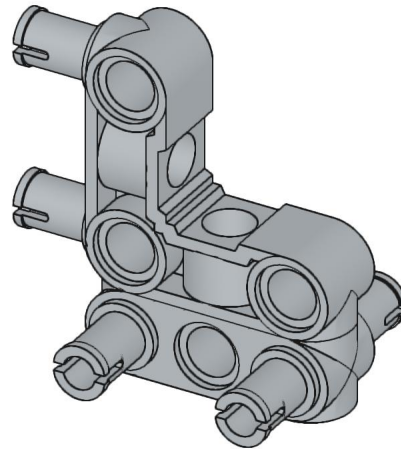
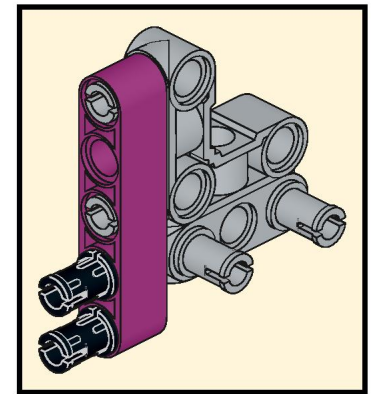
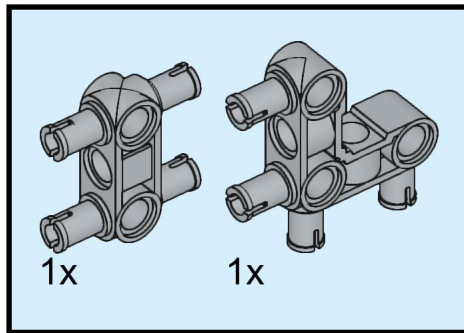
20



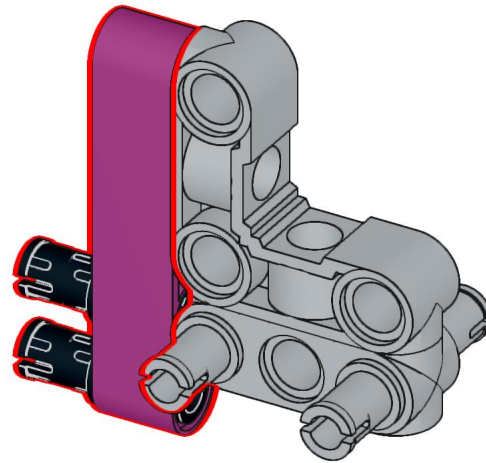
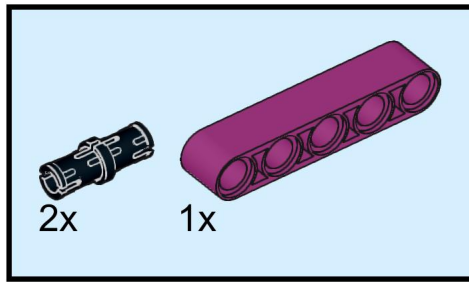
21



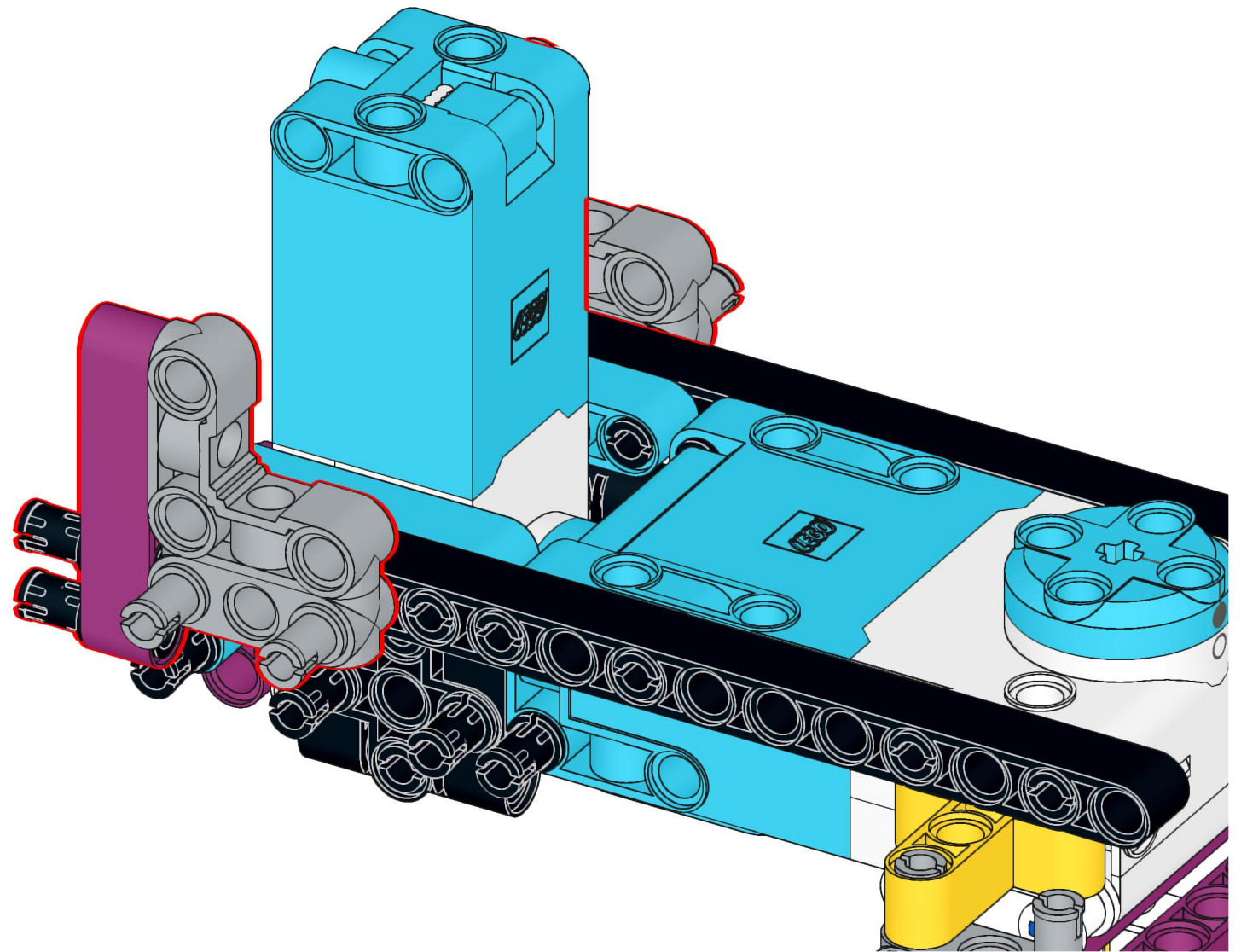
22



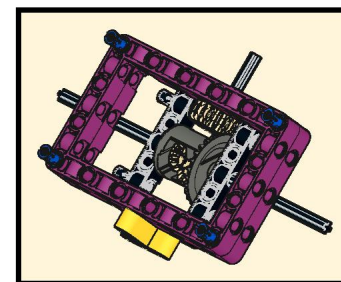
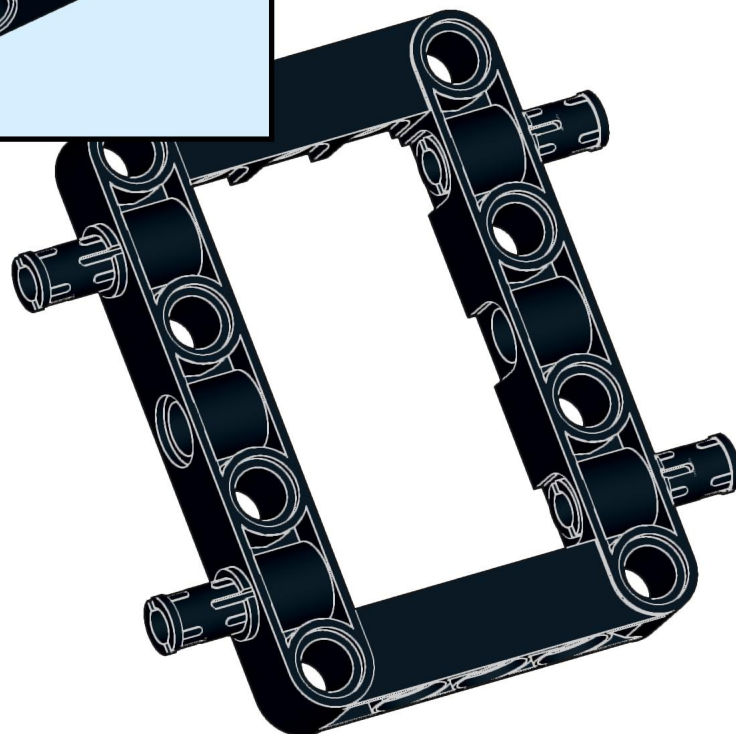
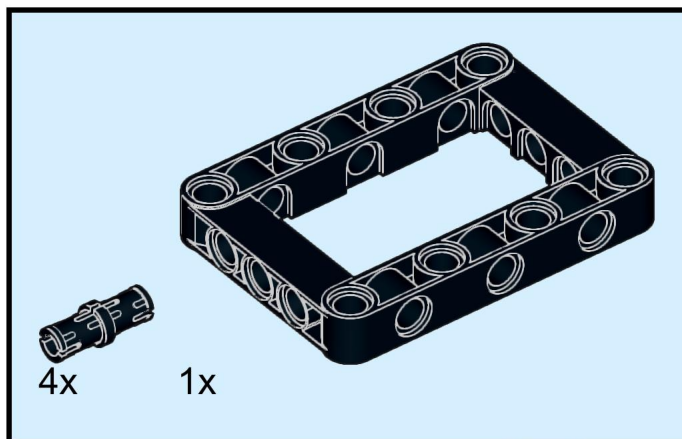
23



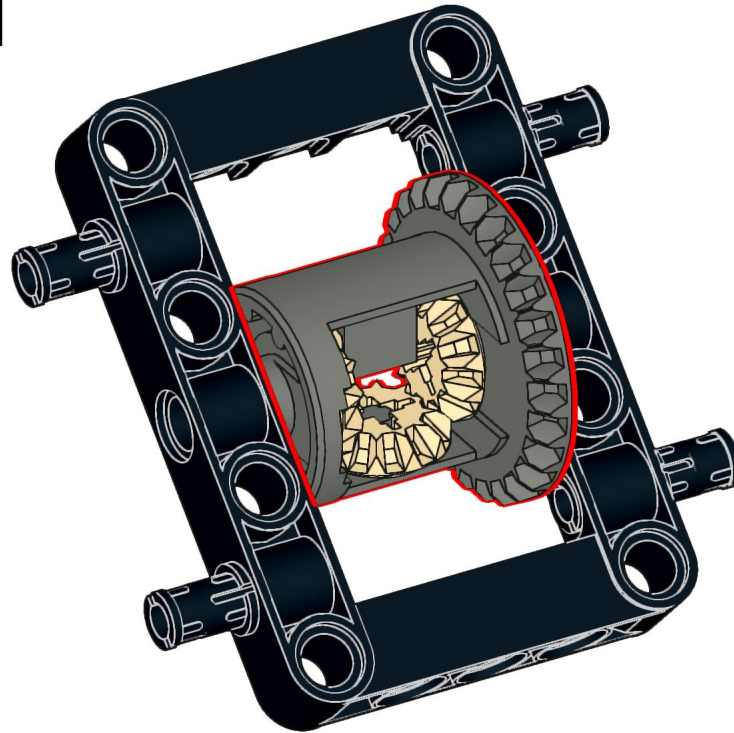
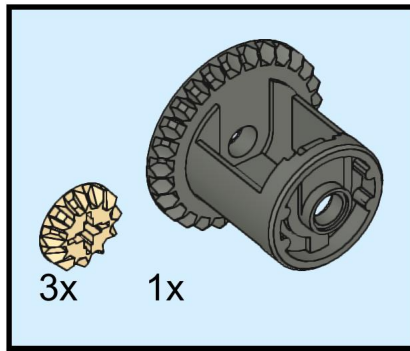
24



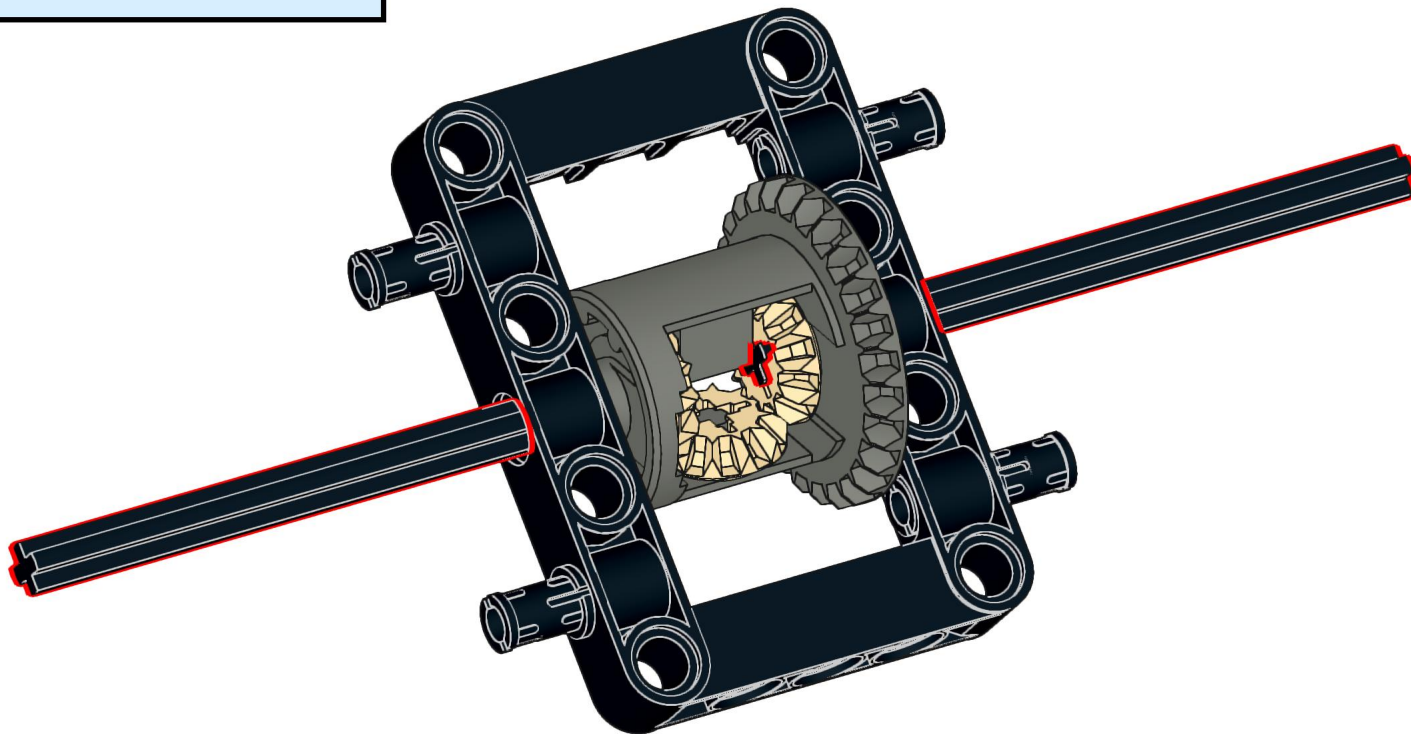
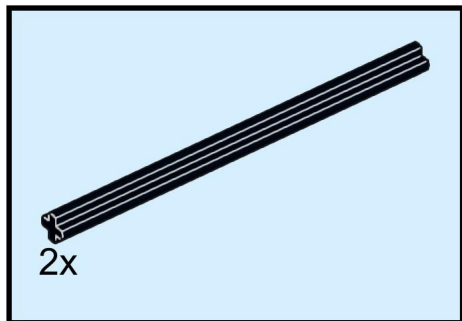
25



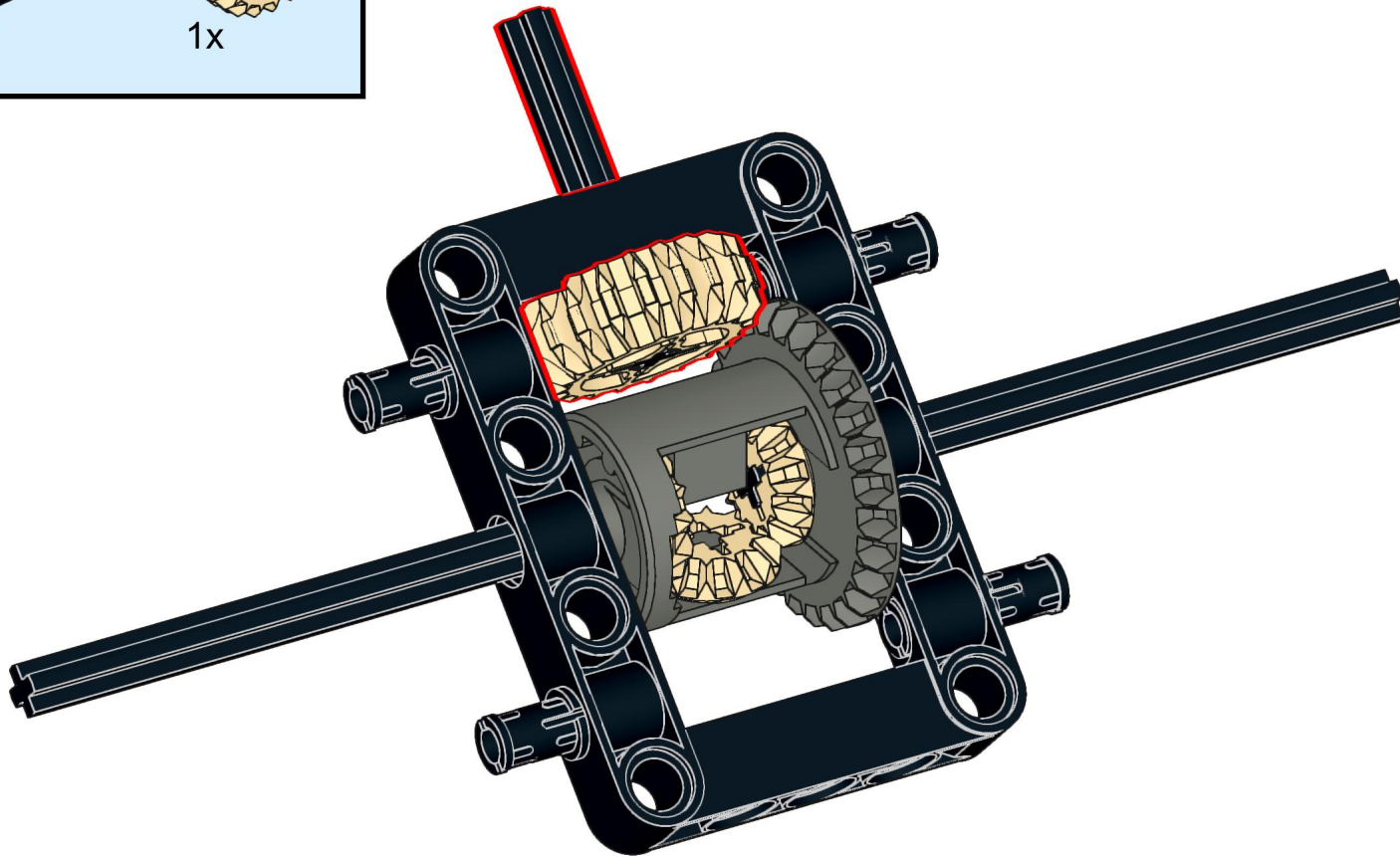
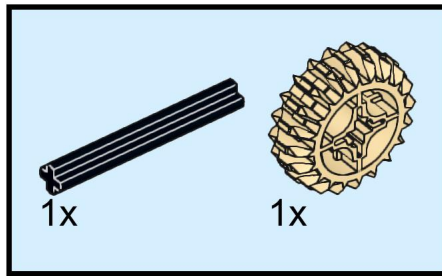
26



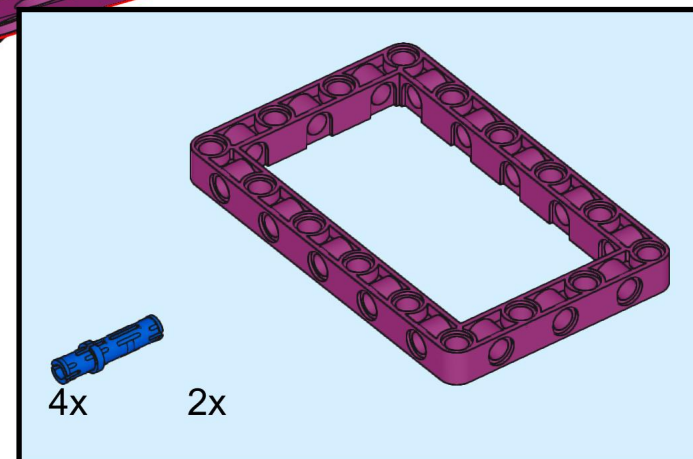
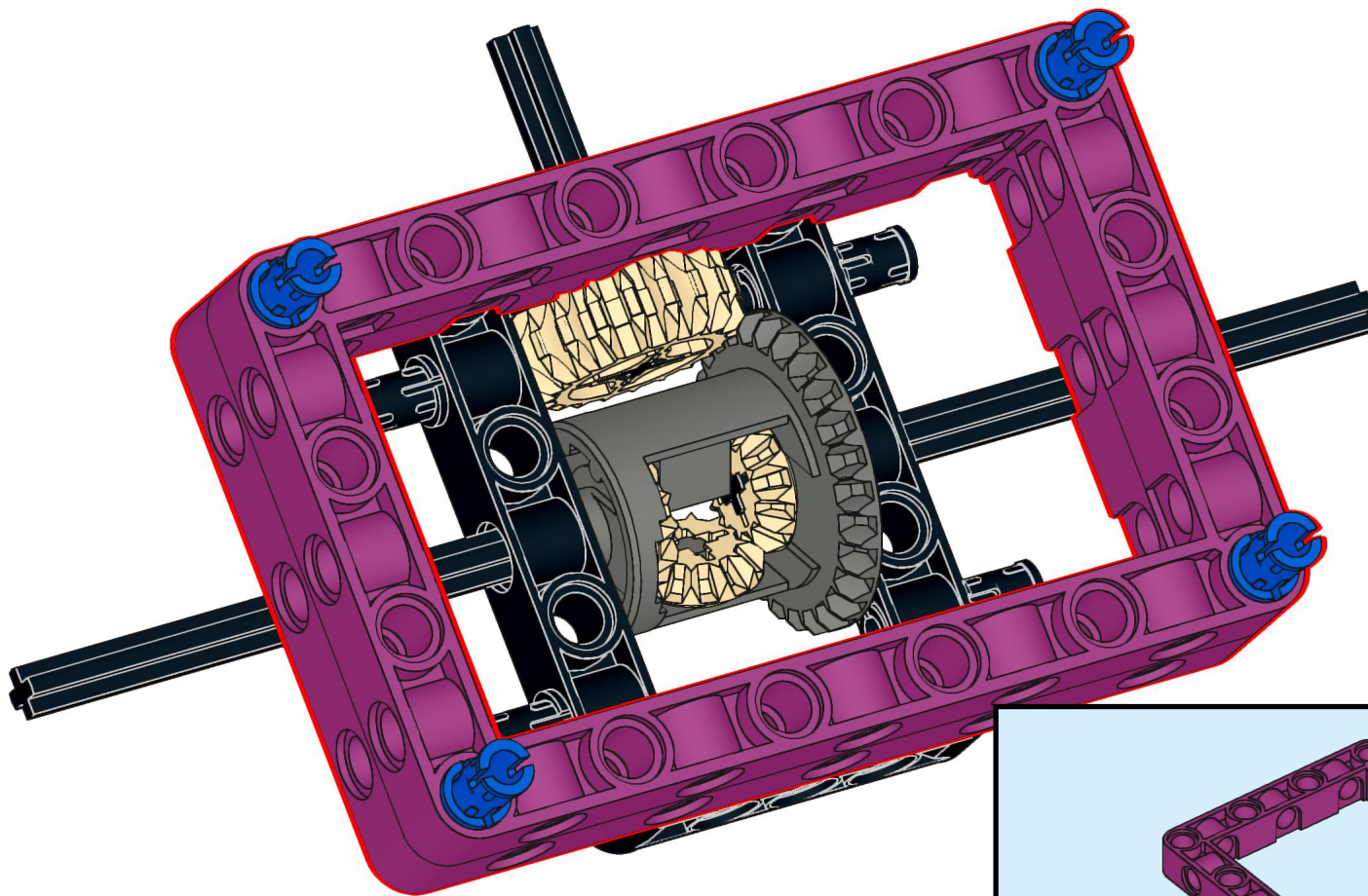
27



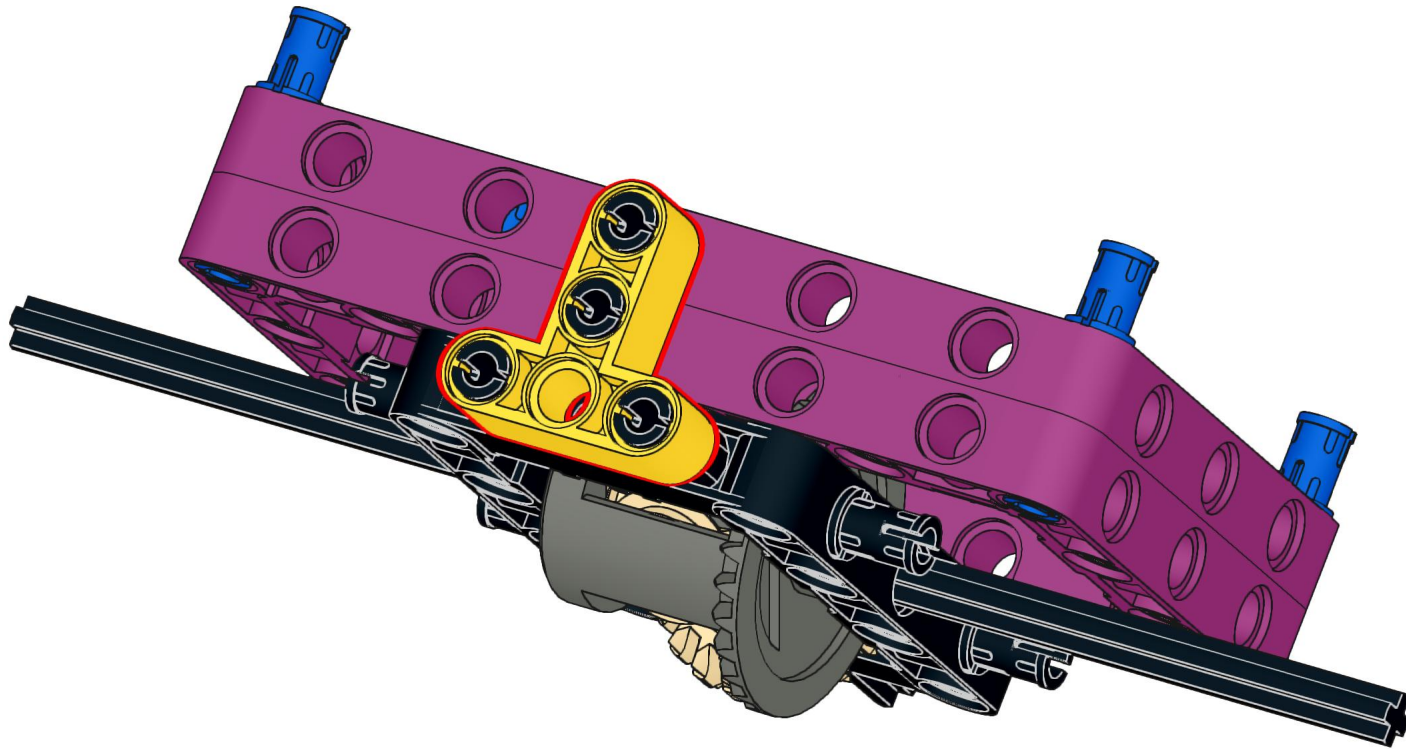
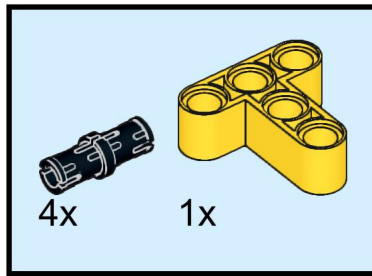
28

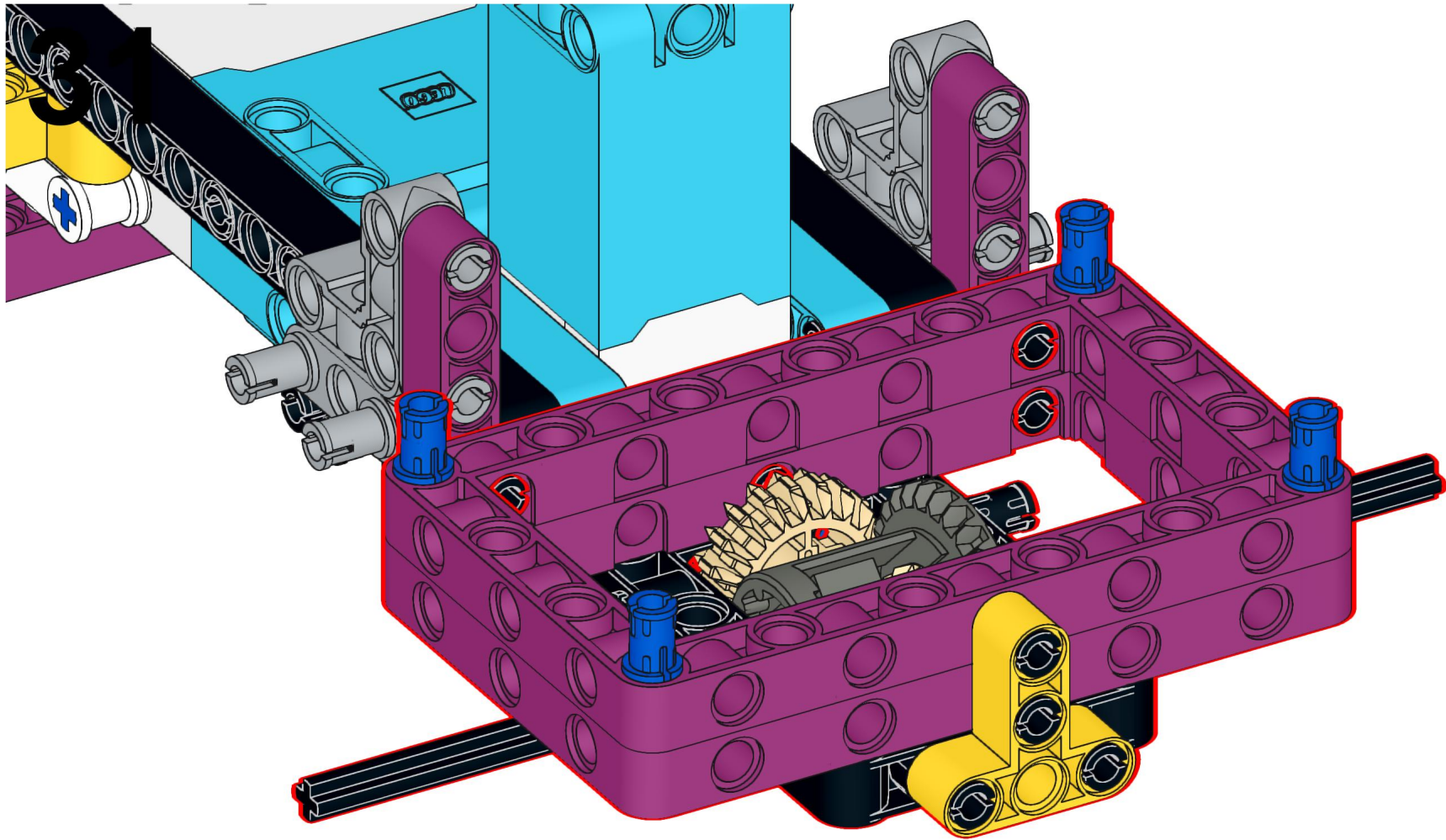


29

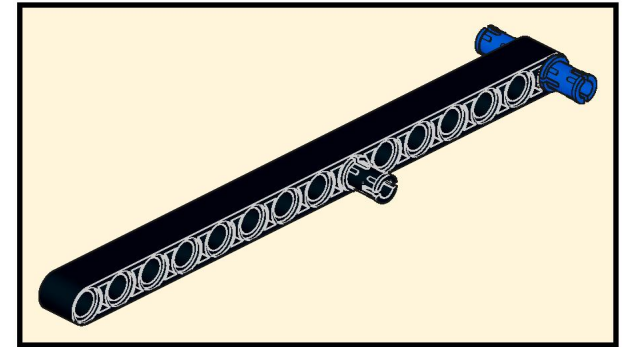
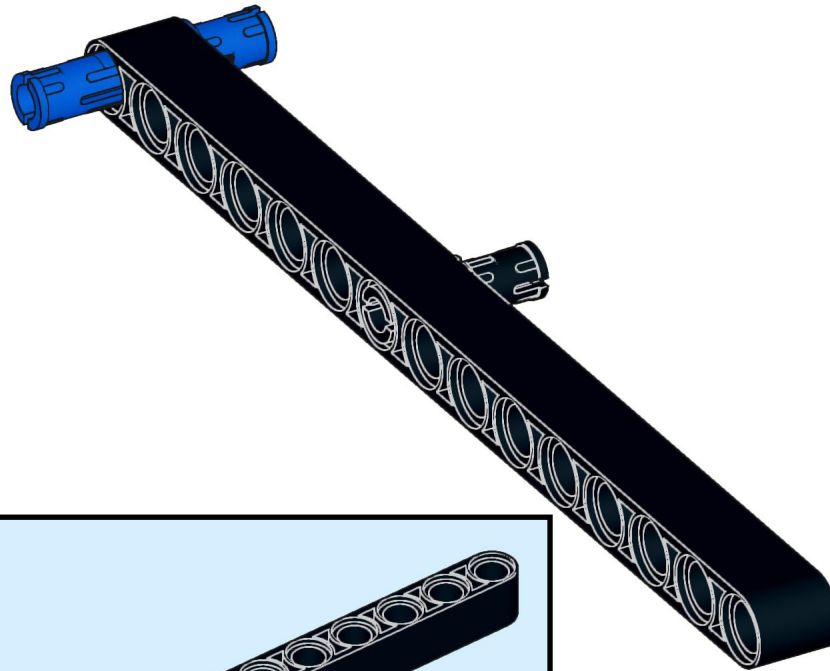


30

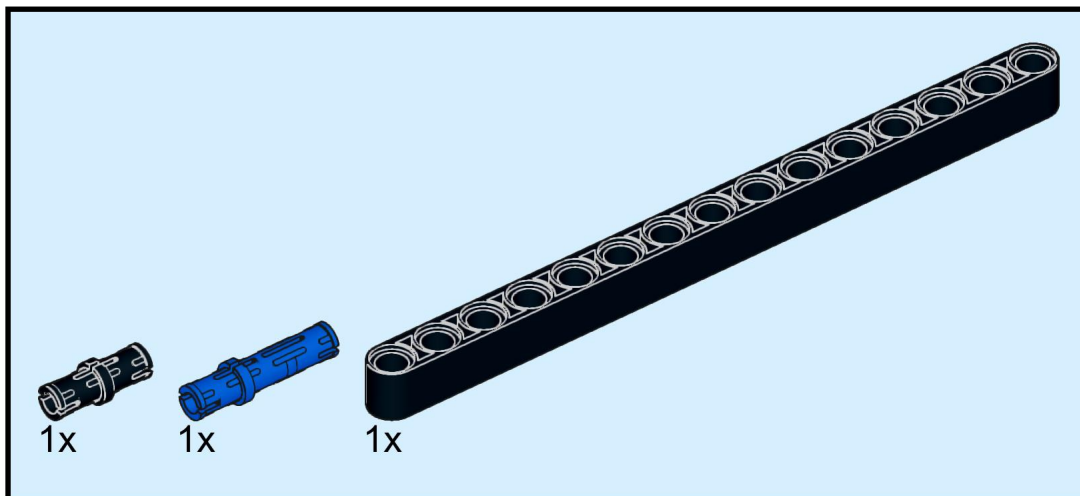




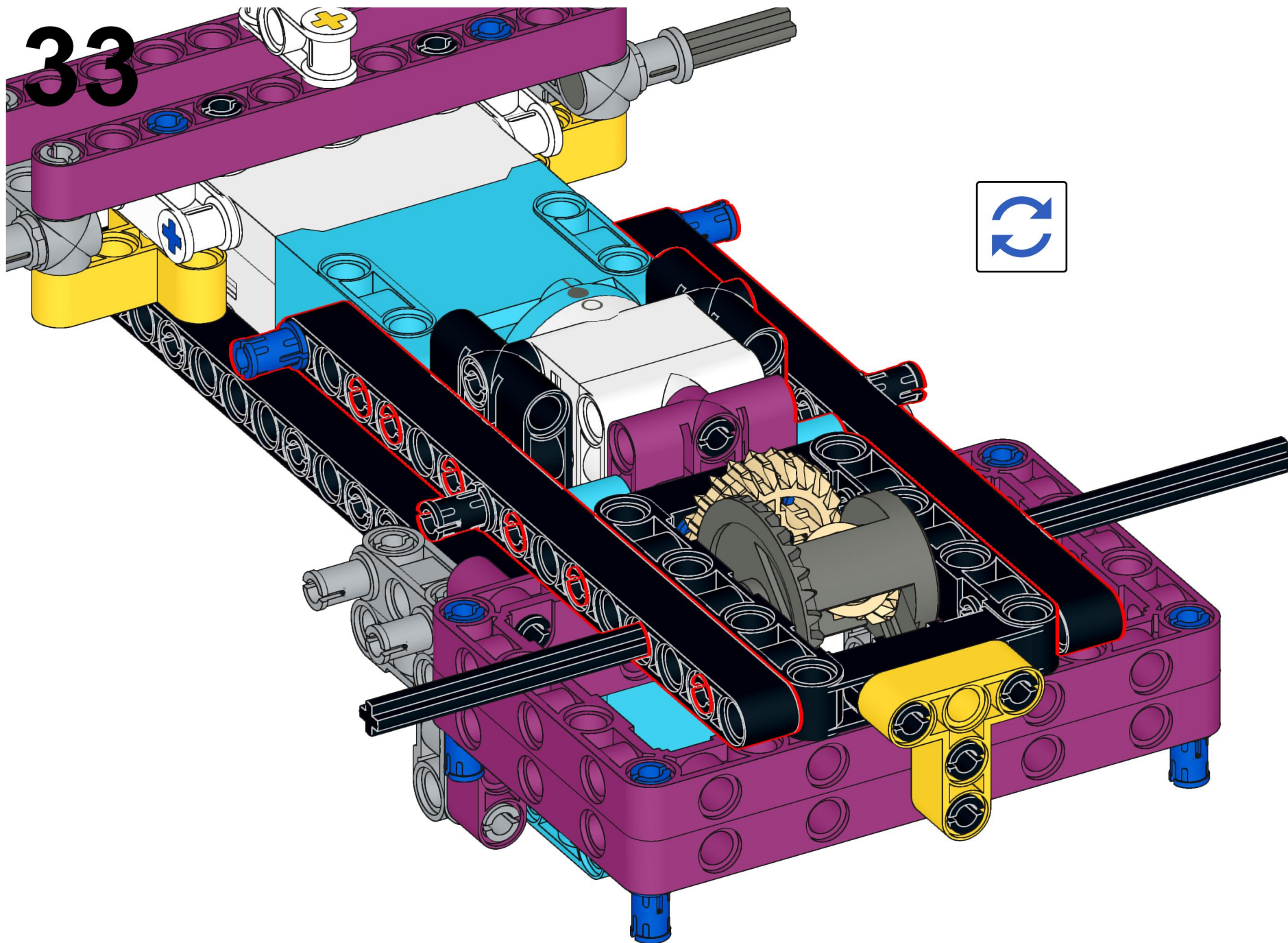
32



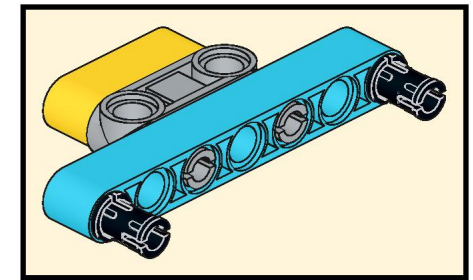
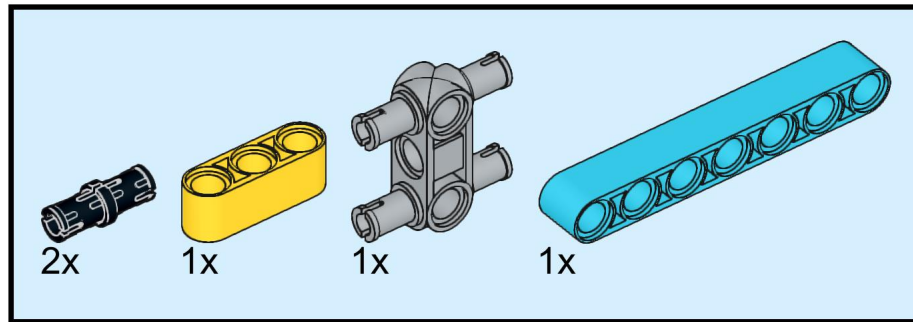
2x



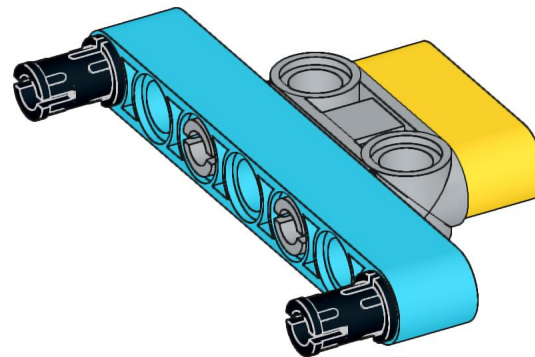
33

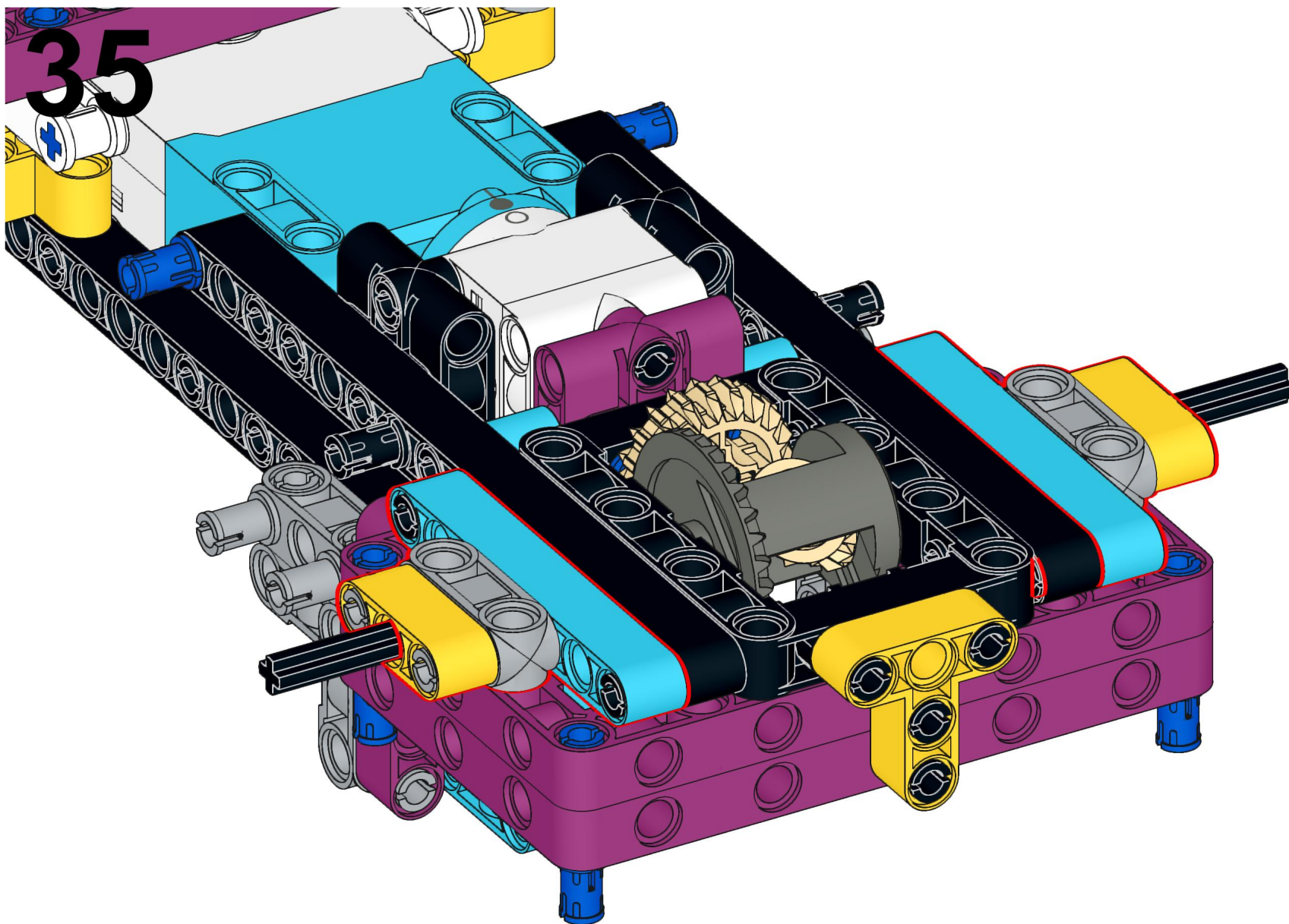


34

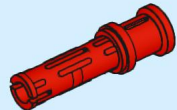


2x

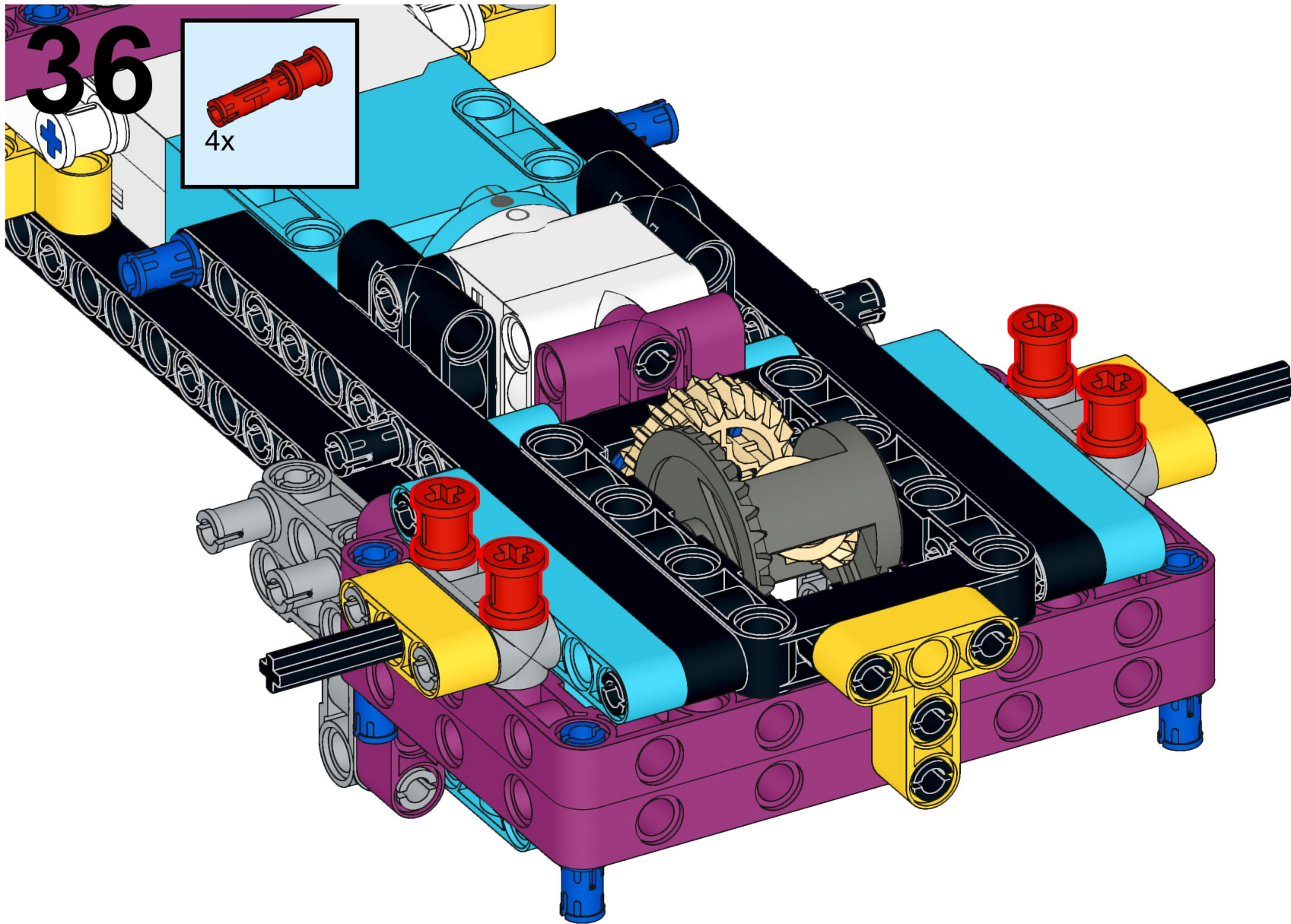




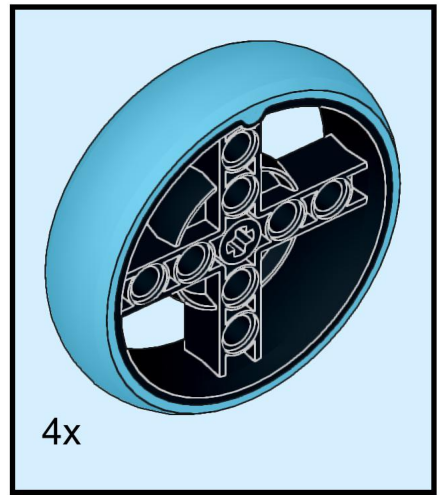
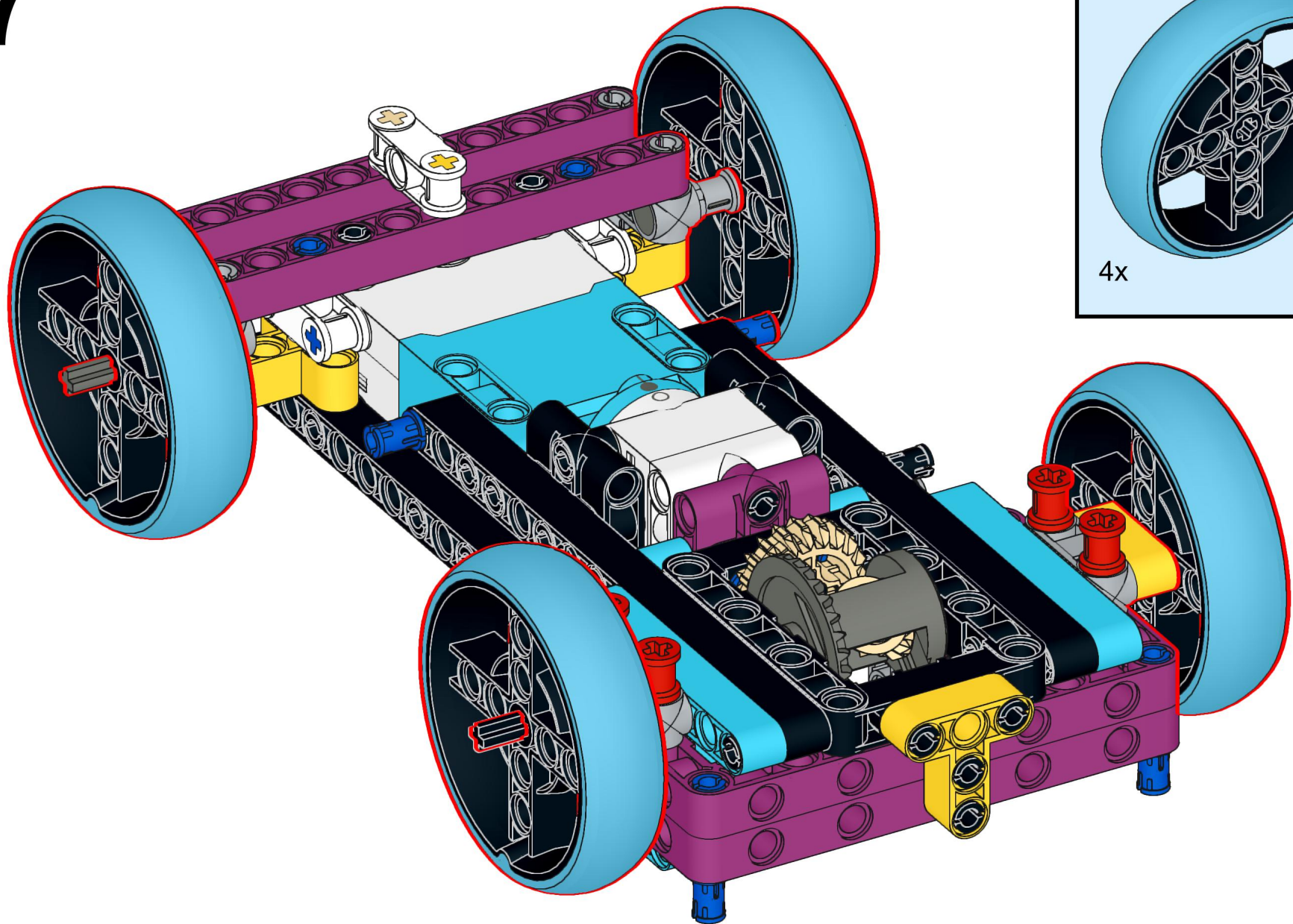
36



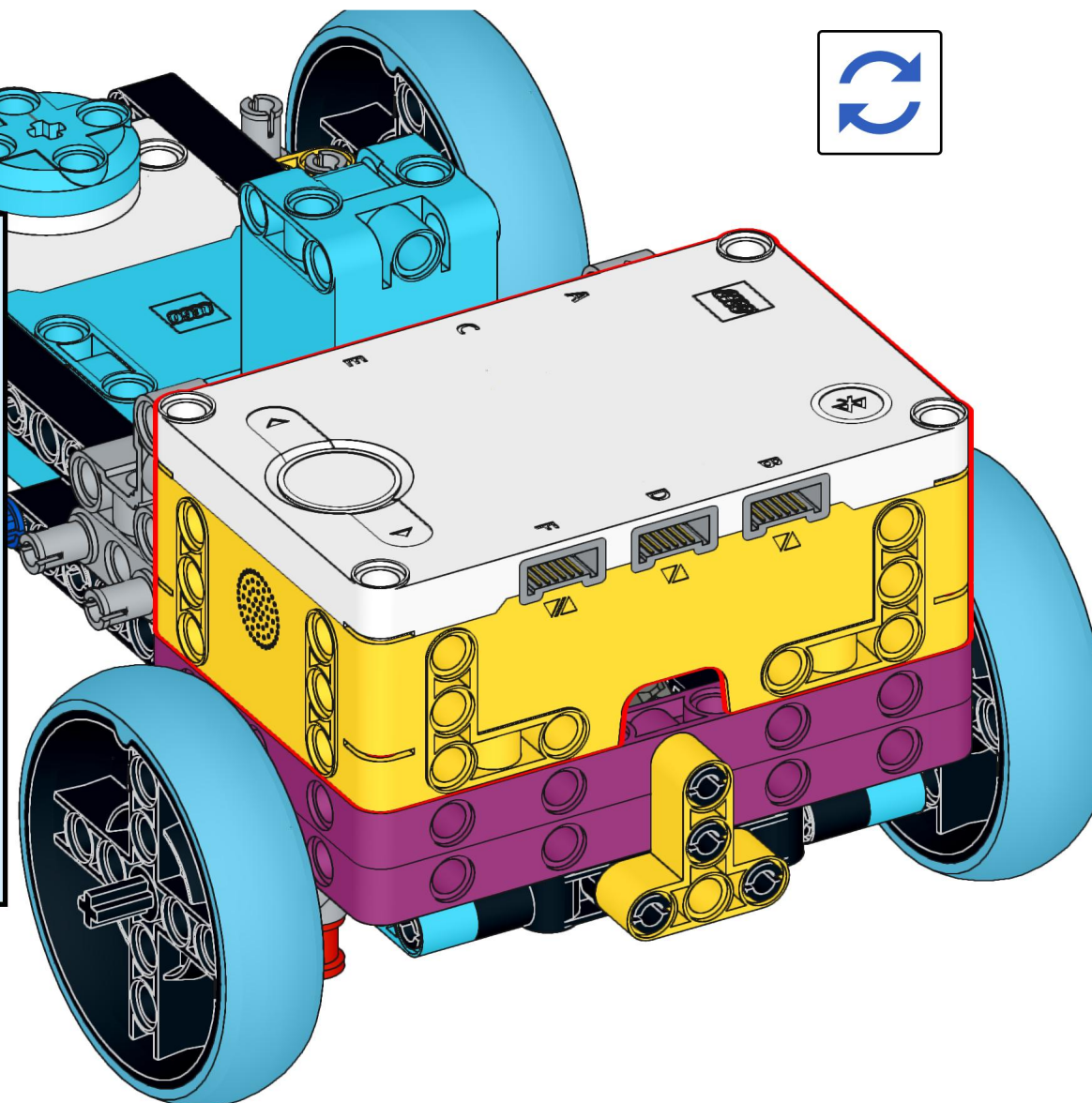
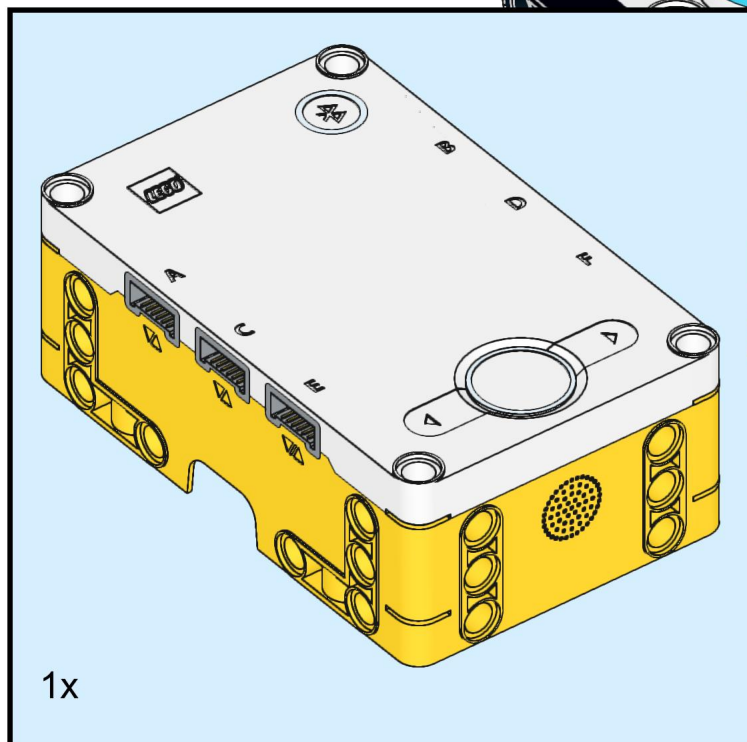
4x



37



38



39

